



การประยุกต์ใช้โปรแกรมจำลองทางคณิตศาสตร์

ในการคำนวณหาผลลัพธ์ของอนุกรมฟูเรียร์

จัดทำโดย

นายกิตติธัช ดอกบัว รหัสนิต 68109010166

นายพิเชษฐ ผลสะอาด รหัสนิต 68109010185

นายณภัส สุรเกียรติชัย รหัสนิต 68109010440

นายวชิรวิทย์ ประสิทธิ์เกตุ รหัสนิต 68109010442

เสนอ

อ.ดร. วชิรยงยศ ทิมาบุตร

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CPE 104 Mathematics for computer Engineering

คณะวิศวกรรมศาสตร์ สาขาวิศวกรรมคอมพิวเตอร์

ภาคการศึกษาที่ 2 ปีการศึกษา 2568

มหาวิทยาลัยศรีนครินทรวิโรฒ

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อศึกษาและพัฒนาโปรแกรมสำหรับคำนวณอนุกรมฟูเรียร์ (Fourier Series) ด้วยภาษา Python โดยมีวัตถุประสงค์เพื่อทำความเข้าใจแนวคิดทางคณิตศาสตร์ในการแปลงฟังก์ชันให้อยู่ในรูปของฟังก์ชัน sine และ cosine รวมถึงศึกษาพฤติกรรมการลู่เข้า (convergence) ของอนุกรมฟูเรียร์เมื่อเพิ่มจำนวนพจน์

โปรแกรมนี้มุ่งเน้นการประยุกต์ใช้ความรู้ด้านคณิตศาสตร์เชิงวิเคราะห์ร่วมกับการเขียนโปรแกรมคอมพิวเตอร์ เพื่อสร้างเครื่องมือที่สามารถคำนวณและแสดงผลอนุกรมฟูเรียร์ได้อย่างเป็นระบบ โดยโปรแกรมที่พัฒนาขึ้นสามารถรับฟังก์ชันจากผู้ใช้ คำนวณค่าสัมประสิทธิ์ และแสดงผลทั้งในรูปแบบสมการและกราฟเพื่อเปรียบเทียบกับฟังก์ชันต้นฉบับ นอกจากนี้ โปรแกรมยังช่วยให้ผู้ใช้สามารถมองเห็นภาพรวมของการประมาณค่าด้วยอนุกรมฟูเรียร์ได้อย่างชัดเจน ซึ่งช่วยเสริมความเข้าใจในเนื้อหาทางคณิตศาสตร์และการประยุกต์ใช้ในทางวิศวกรรมและวิทยาศาสตร์

ในการจัดทำรายงานฉบับนี้ ผู้จัดทำขอขอบพระคุณอาจารย์ผู้สอนที่ได้ให้คำแนะนำ แนวทาง และความรู้ที่เป็นประโยชน์อย่างยิ่งตลอดการดำเนินงาน ซึ่งช่วยให้โปรแกรมนี้สามารถสำเร็จลุล่วงไปได้ด้วยดี ผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจศึกษาเกี่ยวกับอนุกรมฟูเรียร์ และสามารถนำไปต่อยอดในการเรียนรู้หรือพัฒนาโปรแกรมในระดับที่สูงขึ้นต่อไป

คณะผู้จัดทำ

สารบัญ

เรื่อง	หน้า
คำนำ	ก
สารบัญ	ข
ที่มาและความสำคัญ	1
วัตถุประสงค์	2
ขอบเขตของแอปพลิเคชัน และประโยชน์ที่คาดว่าจะได้รับ	2
ทฤษฎี และหลักการของอนุกรมฟูเรียร์	3
เครื่องมือและเทคโนโลยีที่ใช้	6
การออกแบบและหลักการทำงานของโปรแกรม	10
การเขียน Code ของโปรแกรม	14
การจำลอง และทดสอบโจทย์ตัวอย่าง	34
สรุปการดำเนินงาน และข้อจำกัดของโปรแกรม	50
ข้อเสนอแนะในการพัฒนาต่อ	51
บรรณานุกรม	52

ที่มาและความสำคัญ

ในปัจจุบัน การวิเคราะห์สัญญาณและฟังก์ชันทางคณิตศาสตร์มีบทบาทสำคัญอย่างมากในหลายสาขา เช่น วิศวกรรมคอมพิวเตอร์ การสื่อสาร และการประมวลผลข้อมูล โดยเฉพาะสัญญาณที่มีลักษณะซับซ้อนหรือไม่เป็นรูปแบบพื้นฐาน ซึ่งยากต่อการวิเคราะห์โดยตรง อนุกรมฟูเรียร์ (Fourier Series) เป็นเครื่องมือทางคณิตศาสตร์ที่ช่วยแปลงฟังก์ชันหรือสัญญาณที่ซับซ้อนให้สามารถเขียนใหม่ในรูปของผลรวมของฟังก์ชันไซน์และโคไซน์ ซึ่งเป็นฟังก์ชันพื้นฐานที่เข้าใจและวิเคราะห์ได้ง่ายกว่า แนวคิดนี้ช่วยให้สามารถศึกษาองค์ประกอบความถี่ของสัญญาณ และทำให้การแก้ปัญหาทางวิศวกรรมและวิทยาศาสตร์มีประสิทธิภาพมากขึ้น อย่างไรก็ตาม เนื้อหาเกี่ยวกับอนุกรมฟูเรียร์มักมีความเป็นนามธรรมสูง ทำให้ผู้เรียนจำนวนมากเข้าใจได้ยาก โดยเฉพาะในส่วนของกราฟมองภาพว่าการเพิ่มจำนวนพจน์ส่งผลต่อความใกล้เคียงของกราฟอย่างไร

ดังนั้น โปรแกรมนี้จึงถูกพัฒนาขึ้นเพื่อสร้างเครื่องมือที่ช่วยในการแสดงผลและจำลองกราฟของอนุกรมฟูเรียร์ โดยเน้นให้ผู้ใช้งานสามารถมองเห็นความสัมพันธ์ระหว่างจำนวนพจน์กับความแม่นยำของกราฟได้อย่างชัดเจน ซึ่งจะช่วยเสริมสร้างความเข้าใจเชิงภาพ (visual understanding) และทำให้การเรียนรู้มีประสิทธิภาพมากยิ่งขึ้น

นอกจากนี้ โปรแกรมยังถูกพัฒนาด้วยภาษา Python เนื่องจากเป็นภาษาที่มีความเรียบง่าย เข้าใจง่าย และเหมาะสำหรับผู้เริ่มต้น อีกทั้งยังมีไลบรารีด้านคณิตศาสตร์และการแสดงผลกราฟที่ทรงพลัง เช่น NumPy และ Matplotlib ซึ่งช่วยให้สามารถคำนวณและแสดงผลอนุกรมฟูเรียร์ได้อย่างมีประสิทธิภาพและรวดเร็ว รวมถึงสามารถปรับปรุงและต่อยอดโปรแกรมได้สะดวกในอนาคต

วัตถุประสงค์

1. เพื่อพัฒนาโปรแกรมสำหรับคำนวณ Fourier Series ด้วยภาษา Python
2. เพื่อช่วยให้เข้าใจแนวคิดการแปลงฟังก์ชันให้อยู่ในรูป sine และ cosine เพื่อศึกษาการลู่เข้า (convergence) ของ Fourier Series
3. เพื่อแสดงผลเชิงภาพผ่านกราฟ เปรียบเทียบฟังก์ชันจริงกับค่าประมาณ จาก Fourier Series

ขอบเขตของโปรแกรม (Scope)

ใช้คำนวณ Fourier Series สำหรับฟังก์ชันหนึ่งตัวแปร $f(x)$ รองรับทั้ง ฟังก์ชันทั่วไปแบบ (Single) และ ฟังก์ชันแบบ Piecewise

กำหนดช่วงเพื่อสร้างฟังก์ชันคาบ (Periodic Extension) แสดงผลทั้งสมการ Fourier และ กราฟ เปรียบเทียบกับฟังก์ชันจริง รวมถึงใช้การคำนวณแบบเชิงตัวเลข (Numerical Integration) ในการหาค่า สัมประสิทธิ์

ประโยชน์ที่คาดว่าจะได้รับ

1. เข้าใจหลักการของ Fourier Series ได้ชัดเจนยิ่งขึ้น
2. สามารถนำไปประยุกต์ใช้ในการวิเคราะห์สัญญาณหรือระบบต่าง ๆ ได้
3. ฝึกทักษะการเขียนโปรแกรมและการออกแบบ GUI ด้วย Python เห็นภาพการทำงานของคณิตศาสตร์ ผ่านกราฟ ทำให้เรียนรู้ได้ง่ายขึ้น

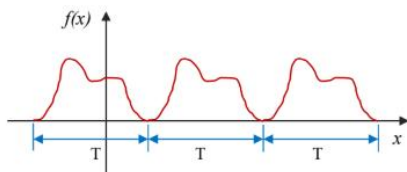
ทฤษฎีและหลักการของอนุกรมฟูรีเยร์

นิยามและคุณสมบัติ

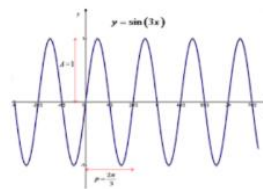
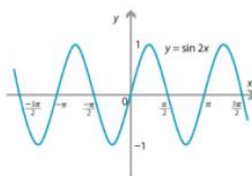
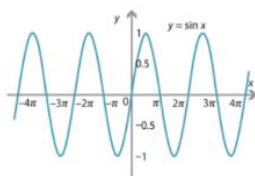
1. $f(x)$ เป็นฟังก์ชันคาบก็ต่อเมื่อมีจำนวนจริงบวก T ที่ทำให้ $f(x + T) = f(x)$ สำหรับทุก x

และเรียก T ว่าคาบ (period) ของ $f(x)$

กราฟของฟังก์ชันเป็นคาบที่มีคาบเท่ากับ T จะมีลักษณะซ้ำกันในช่วงความยาว T



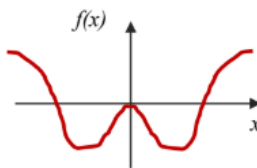
ฟังก์ชัน $\sin(nx)$ และ $\cos(nx)$ มีคาบเท่ากับ 2π



2. ฟังก์ชัน $f(x)$ ใดๆจะเป็น ฟังก์ชันคู่ (even function) ก็ต่อเมื่อ $f(-x) = f(x)$ ทุกค่า x

เช่น $f(x) = x^2$

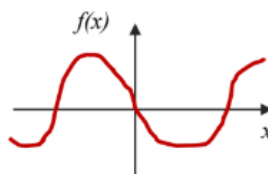
$$f(x) = \cos(nx)$$



3. ฟังก์ชัน $f(x)$ ใดๆจะเป็น ฟังก์ชันคี่ (odd function) ก็ต่อเมื่อ $f(-x) = -f(x)$ ทุกค่า x

เช่น $f(x) = x$

$$f(x) = \sin(nx)$$



4. ในกรณีที่ฟังก์ชัน $f(x)$ ไม่สอดคล้องเงื่อนไขของฟังก์ชันคู่และฟังก์ชันคี่ ฟังก์ชันในลักษณะนี้จะเรียกว่า ไม่เป็นทั้งฟังก์ชันคู่และฟังก์ชันคี่ (Neither even nor odd function)

คุณสมบัติบางประการของฟังก์ชันคู่และฟังก์ชันคี่

$$1. (\text{ฟังก์ชันคู่}) \times (\text{ฟังก์ชันคู่}) = \text{ฟังก์ชันคู่}$$

$$2. (\text{ฟังก์ชันคู่}) \times (\text{ฟังก์ชันคี่}) = \text{ฟังก์ชันคี่}$$

$$3. (\text{ฟังก์ชันคี่}) \times (\text{ฟังก์ชันคี่}) = \text{ฟังก์ชันคู่}$$

$$4. (\text{ฟังก์ชันคู่}) \pm (\text{ฟังก์ชันคู่}) = \text{ฟังก์ชันคู่}$$

$$5. (\text{ฟังก์ชันคี่}) \pm (\text{ฟังก์ชันคี่}) = \text{ฟังก์ชันคี่}$$

$$6. \text{ถ้า } f(x) \text{ เป็นฟังก์ชันคู่แล้ว } \int_{-c}^c f(x)dx = 2 \int_0^c f(x)dx$$

$$7. \text{ถ้า } f(x) \text{ เป็นฟังก์ชันคี่แล้ว } \int_{-c}^c f(x)dx = 0$$

อนุกรมฟูรีเยร์ (Fourier Series) เป็นเครื่องมือที่ใช้วิเคราะห์ผลประกอบทางด้านความถี่ของฟังก์ชันต่อเนื่องที่มีคาบเท่ากับ $2L$ และ x เป็นตัวแปรที่ต่อเนื่องแล้ว $f(x)$ สามารถเขียนให้อยู่ในรูปอนุกรมฟูรีเยร์ตรีโกณมิติได้ดังนี้

$$f(x) = a_0 + \sum_{n=1}^{\infty} [a_n \cos(\frac{n\pi x}{L}) + b_n \sin(\frac{n\pi x}{L})]$$

a_0, a_n, b_n เป็นค่าสัมประสิทธิ์ของการรวมซึ่งเป็นค่าคงที่สามารถหาได้จาก

$$a_0 = \frac{1}{2L} \int_{-L}^L f(x)dx$$

$$a_n = \frac{1}{L} \int_{-L}^L f(x) \cos\left(\frac{n\pi x}{L}\right) dx$$

$$b_n = \frac{1}{L} \int_{-L}^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx$$

กรณี $f(x)$ เป็นฟังก์ชันคู่หรือฟังก์ชันคี่ ค่าของ a_0, a_n, b_n จะมีค่าตามตาราง

	เงื่อนไข	คุณสมบัติ
ฟังก์ชันคู่	$f(x) = f(-x)$	$b_n = 0$
ฟังก์ชันคี่	$-f(x) = f(-x)$	$a_0 = a_n = 0$

ประโยชน์ของอนุกรมฟูรีเยร์

1. การประมวลผลสัญญาณและสื่อสาร (Signal processing) ใช้แยกแยะองค์ประกอบความถี่ของสัญญาณ ช่วยในการออกแบบตัวกรองสัญญาณ (Filters) เพื่อลดสัญญาณรบกวน (Noise)
2. การบีบอัดข้อมูล (Data Compression) อนุกรมฟูรีเยร์เป็นรากฐานของการแปลงฟูรีเยร์แบบไม่ต่อเนื่อง (DFT) ซึ่งใช้บีบอัดข้อมูลภาพ (JPEG) และเสียง (MP3) โดยการตัดความถี่ที่ไม่สำคัญออก
3. การวิเคราะห์ทางวิศวกรรม ใช้แก้สมการเชิงอนุพันธ์ย่อย (PDE) เช่น การวิเคราะห์การสั่นสะเทือน, การถ่ายเทความร้อน (Heat Equation), และการวิเคราะห์โครงสร้าง
4. การอธิบายสัญญาณคาบ: ช่วยเปลี่ยนคลื่นที่มีรูปทรงซับซ้อน (เช่น สัญญาณสี่เหลี่ยม หรือคลื่นฟันเลื่อย) ให้อยู่ในรูปของผลรวมของคลื่นไซน์ (Sine) และโคไซน์ (Cosine)
5. ฟิสิกส์และคณิตศาสตร์: ใช้ในการวิเคราะห์สมบัติของฟังก์ชันคู่ (Even) และคี่ (Odd) และช่วยให้เข้าใจแนวคิดเกี่ยวกับพีชคณิตเชิงเส้นได้ชัดเจนขึ้น

เครื่องมือและเทคโนโลยีที่ใช้

ภาษาโปรแกรม

- Python

ไลบรารีที่ใช้

- NumPy (คำนวณเชิงตัวเลข)
- Matplotlib (แสดงกราฟ)
- SciPy (อินทิกรัล)
- Tkinter (สร้าง GUI)

โปรแกรมพัฒนา (IDE)

1. Visual Studio Code (ในที่นี้เราจะใช้โปรแกรมนี้นในการรันโปรแกรม)
เป็นโปรแกรมแก้ไขโค้ดที่มีความยืดหยุ่นสูง รองรับหลายภาษา และสามารถติดตั้งส่วนเสริม (Extensions) เพื่อเพิ่มความสามารถ เช่น Python Extension
2. PyCharm
เป็น IDE ที่ออกแบบมาสำหรับภาษา Python โดยเฉพาะ มีระบบช่วยเขียนโค้ด (Auto-completion) และ Debugging ที่มีประสิทธิภาพ

การใช้งาน Visual Studio Code

การใช้งาน Visual Studio Code มีขั้นตอนดังนี้

1. ติดตั้งโปรแกรม VS Code
2. ติดตั้ง Python และ Extensions ที่จำเป็น
3. ติดตั้งไลบรารีด้วยคำสั่ง

```
pip install numpy matplotlib scipy
```

4. เปิดไฟล์โปรแกรม (.py)
5. รันโปรแกรมผ่านปุ่ม Run หรือ Terminal

ข้อดีของ VS Code:

- ใช้งานง่ายและรวดเร็ว
- รองรับหลายภาษา
- มีระบบ Extension เพิ่มความสามารถ

การใช้งาน PyCharm

1. ติดตั้งโปรแกรม PyCharm
2. สร้าง Project ใหม่
3. เลือก Python Interpreter
4. เพิ่มไลบรารีที่จำเป็น
5. รันโปรแกรมผ่าน IDE

ข้อดีของ PyCharm:

- มีระบบช่วยเขียนโค้ดอัตโนมัติ
- ตรวจสอบข้อผิดพลาดได้ดี
- เหมาะสำหรับโปรเจกต์ขนาดใหญ่

ความรู้เบื้องต้นเกี่ยวกับ Visual Studio Code

Visual Studio Code (VS Code) เป็นโปรแกรมแก้ไขโค้ด (Code Editor) ที่พัฒนาโดย Microsoft มีความสามารถรองรับหลายภาษาโปรแกรม และสามารถเพิ่มความสามารถได้ผ่าน Extensions จึงเหมาะสำหรับทั้งผู้เริ่มต้นและผู้พัฒนาระดับสูง

1. การติดตั้งโปรแกรม

1. ดาวน์โหลดโปรแกรมจากเว็บไซต์ทางการของ VS Code
2. ติดตั้งตามขั้นตอน (Next → Install → Finish)
3. เปิดโปรแกรมหลังติดตั้งเสร็จ

2. การติดตั้ง Python และส่วนเสริม

2.1 ติดตั้ง Python

- ดาวน์โหลด Python จาก python.org
- ตรวจสอบการติดตั้งโดยพิมพ์คำสั่งใน Terminal:

```
python --version
```

2.2 ติดตั้ง Extension ใน VS Code

ไปที่เมนู Extensions (ไอคอนสี่เหลี่ยม) แล้วติดตั้ง:

- Python (โดย Microsoft)
- Pylance (ช่วย autocomplete และตรวจ error)

3. การสร้างและเปิดโปรเจกต์

1. เปิด VS Code
2. ไปที่เมนู File → Open Folder
3. เลือกโฟลเดอร์สำหรับเก็บโปรเจกต์
4. สร้างไฟล์ใหม่ เช่น fourier_app.py

4. การติดตั้งไลบรารีที่จำเป็น

เปิด Terminal ใน VS Code (กด `Ctrl + ``) แล้วพิมพ์:

```
pip install numpy matplotlib scipy
```

ไลบรารีที่ติดตั้ง:

- numpy → คำนวณตัวเลข
- matplotlib → วาดกราฟ
- scipy → อินทิกรัล

5. การรันโปรแกรม

สามารถรันได้ 2 วิธี

วิธีที่ 1:

กดปุ่ม ▶ Run ที่มุมขวาบน

วิธีที่ 2:

ใช้ Terminal

```
python fourier_app.py
```

การออกแบบและหลักการทำงานของโปรแกรม

1. การออกแบบส่วนติดต่อผู้ใช้ (GUI Design Analysis)

ส่วนติดต่อผู้ใช้ถูกออกแบบภายใต้หลักการความเรียบง่าย โดยสามารถแบ่งโครงสร้างวิดเจ็ต (Widgets) ได้ดังนี้:

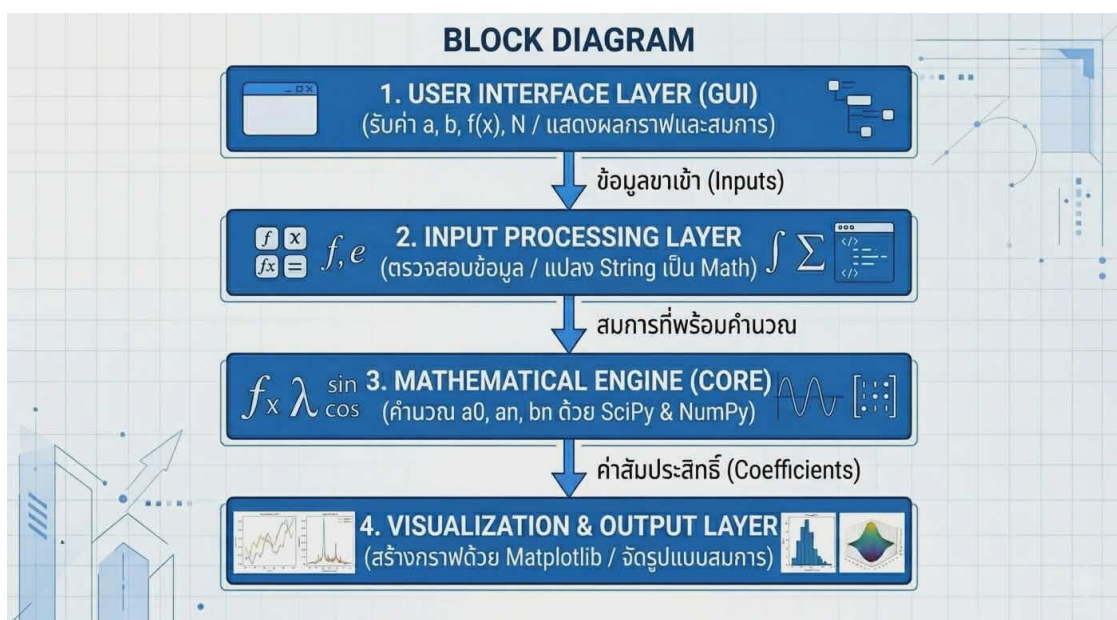
- **ส่วนหัวข้อมูลการตั้งค่า (Header):** ปรากฏข้อความ "1. Interval & Function Configuration" เพื่อจัดกลุ่มพารามิเตอร์นำเข้าให้เป็นระเบียบ
- **ส่วนรับข้อมูล (Configuration Widgets):**
 - วิดเจ็ต Entry สำหรับขอบเขต: ขอบเขตซ้าย (a) และ ขอบเขตขวา (b) เพื่อกำหนดโดเมนของฟังก์ชัน
 - วิดเจ็ต Entry สำหรับฟังก์ชัน: $f_1(x)$ และ $f_2(x)$ รองรับการป้อนสตริงฟังก์ชันคณิตศาสตร์ เช่น $-x$
 - วิดเจ็ต Entry สำหรับความละเอียด: จำนวนพจน์ (N) เพื่อกำหนดระดับความละเอียดของอนุกรม
 - วิดเจ็ต Entry สำหรับจุดแบ่ง (Split): สำหรับระบุค่าพารามิเตอร์ที่ใช้เป็นจุดรอยต่อ (Point of discontinuity) ระหว่างฟังก์ชัน $f_1(x)$ และ $f_2(x)$ เพื่อให้ระบบทำการแยกอินทิเกรตตามโดเมนที่ผู้ใช้กำหนดได้อย่างแม่นยำ
- **ส่วนการเลือกโหมด (Selection Mode):** วิดเจ็ต Dropdown menu (Combobox) แสดงตัวเลือกโหมด ซึ่งมี 2 โหมด คือ "Single" และ "Piecewise"
- **ส่วนควบคุมการทำงาน (Control Logic):** ปุ่ม "Calculate & Plot (3 Periods)" ทำหน้าที่เป็น Event Trigger เพื่อเริ่มกระบวนการคำนวณทางคณิตศาสตร์และอัปเดตการแสดงผล
- **พื้นที่แสดงผลกราฟ (Output Canvas):** พื้นที่ว่างด้านล่างถูกจัดเตรียมไว้สำหรับฝังหน้าต่างกราฟจาก Matplotlib เพื่อการตอบสนองแบบ Interactive

2. การออกแบบเชิงระบบปฏิบัติการ (Operating System Design)

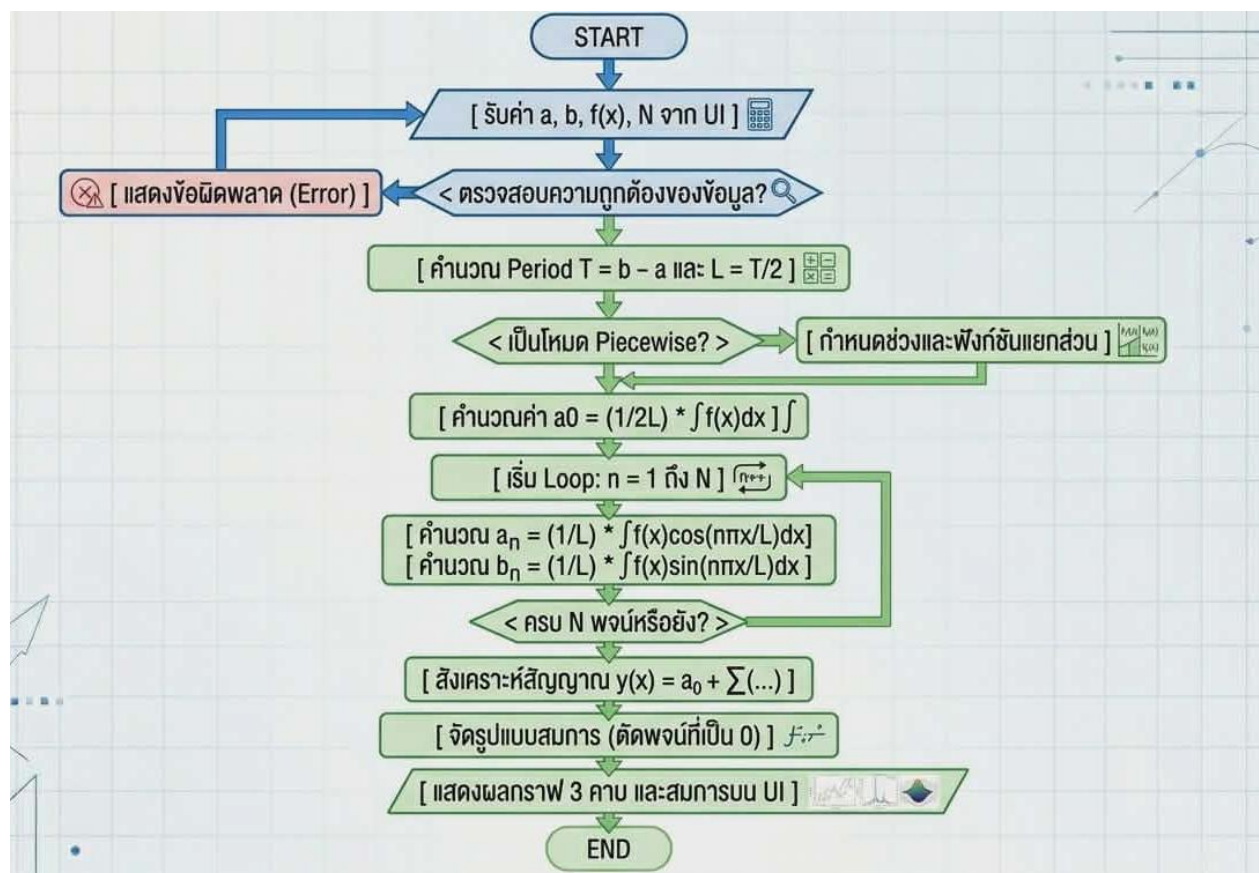
ในการพัฒนาโปรแกรม เราเน้นการออกแบบสถาปัตยกรรมแบบแยกส่วน (Layered Architecture) เพื่อให้การจัดการข้อมูลและการคำนวณคณิตศาสตร์มีความแม่นยำและตรวจสอบได้ง่าย โดยมีรายละเอียดดังนี้

Block Diagram ของระบบ ระบบถูกแบ่งออกเป็น 4 ชั้นสื่อสารหลัก (Layers) ตามโครงสร้างดังนี้

1. User Interface Layer: ทำหน้าที่เป็นด่านหน้าในการรับค่าจากผู้ใช้ (GUI) ผ่านทางช่องป้อนข้อมูล และปุ่มคำสั่ง
2. Input Processing Layer: ทำหน้าที่ตรวจสอบความถูกต้องของข้อมูล (Validation) และแปลงชุดข้อความ (String) ให้เป็นรูปแบบที่คอมพิวเตอร์เข้าใจได้
3. Mathematical Engine (Fourier Core): หัวใจหลักของระบบที่ใช้การอินทิเกรตเชิงตัวเลข (Numerical Integration) เพื่อหาค่าสัมประสิทธิ์ a_0, a_n, b_n ตามทฤษฎีของฟูรีเยร์
4. Visualization & Output Layer: ส่วนการประมวลผลข้อมูลดิบให้กลายเป็นรูปภาพ (Graph) และข้อความสมการที่สามารถอ่านเข้าใจได้



Flowchart การทำงานของโปรแกรม



หลักการทำงานของโปรแกรม Fourier Series Analysis

ผู้ใช้งานจะเริ่มต้นโดยการกำหนดฟังก์ชันที่ต้องการวิเคราะห์ในโปรแกรม ซึ่งสามารถเลือกได้ทั้งฟังก์ชันแบบทั่วไป (Single function) หรือฟังก์ชันแบบแบ่งช่วง (Piecewise function) จากนั้นผู้ใช้งานจะกำหนดช่วงของฟังก์ชัน $[a, b]$ เพื่อใช้ในการสร้างฟังก์ชันแบบคาบ (Periodic extension) รวมถึงในการคำนวณ Fourier Series โปรแกรมจะนำฟังก์ชันที่ผู้ใช้งานกำหนดไปทำการคำนวณค่าสัมประสิทธิ์ของ Fourier Series ได้แก่ a_0, a_n, b_n โดยใช้วิธีการอินทิกรัลเชิงตัวเลข (Numerical Integration) เพื่อประมาณค่าที่แม่นยำ

เมื่อทำการคำนวณเรียบร้อยแล้ว โปรแกรมจะนำผลลัพธ์ไปสร้างเป็นอนุกรมฟูเรียร์ และแสดงผลในรูปแบบสมการเพื่อให้ผู้ใช้งานสามารถเข้าใจโครงสร้างของฟังก์ชันที่ถูกประมาณได้อย่างชัดเจน

พร้อมกันนี้ โปรแกรมจะทำการแสดงผลในรูปแบบกราฟ โดยเปรียบเทียบระหว่างฟังก์ชันต้นฉบับและฟังก์ชันที่ได้จาก Fourier Series เพื่อให้ผู้ใช้งานสามารถสังเกตพฤติกรรมการลู่เข้า (convergence) ได้อย่างชัดเจนเมื่อเพิ่มจำนวนพจน์ N

ผู้ใช้งานยังสามารถปรับจำนวนพจน์ของอนุกรมได้ตามต้องการ เพื่อดูความแม่นยำของการประมาณค่าในแต่ละระดับ และสังเกตผลกระทบของจำนวนพจน์ที่มีต่อความใกล้เคียงกับฟังก์ชันจริง

การเขียน Code ของโปรแกรม

การพัฒนาโปรแกรม ใช้สถาปัตยกรรมเชิงวัตถุ (Object-Oriented Programming - OOP) เพื่อรวบรวมฟังก์ชันการทำงานที่ซับซ้อนให้เป็นระบบเดียว โดยสามารถแยกวิเคราะห์การทำงานของแต่ละเมธอด (Method) ได้ดังนี้

1. การเตรียมสภาพแวดล้อมและ Library (Environment Setup)

ในการพัฒนาโปรแกรมนี้นี้ เป็นการประยุกต์ใช้ Ecosystem ของ Python ในการแก้ปัญหาทางคณิตศาสตร์และการแสดงผลทางคณิตศาสตร์ อย่างเป็นระบบ โดยมีการใช้ไลบรารีเฉพาะทางดังนี้

ไลบรารี	โมดูล/เมธอดสำคัญ	หน้าที่และความรับผิดชอบ
numpy	np.linspace, np.sin, np.cos	จัดการการประมวลผลอาร์เรย์และการคำนวณเวกเตอร์
scipy.integrate	quad	การหาค่าปริพันธ์เชิงตัวเลข (Numerical Integration) เพื่อคำนวณสัมประสิทธิ์
matplotlib	pyplot	การสร้างและจัดการคุณลักษณะของกราฟ (Plotting Logic)
matplotlib.backends	FigureCanvasTkAgg	ทำหน้าที่เป็นตัวกลาง (Bridge) เชื่อมต่อกราฟ Matplotlib เข้ากับเฟรมของ Tkinter
tkinter / ttk	Tk, Entry, Button	การสร้างสถาปัตยกรรม GUI และวิดเจ็ตมาตรฐาน

คำสั่งที่ต้องใช้ในการดาวน์โหลดไลบรารี

```
pip install numpy matplotlib scipy
```

Code ในส่วนการเตรียม Environment ในการเขียนโปรแกรม

การนำเข้า (import) ไลบรารีต่างๆ โปรแกรมนี้มีการนำเข้าไลบรารีที่จำเป็นเพื่อรองรับการทำงานในด้านการคำนวณเชิงตัวเลข การแสดงผล และการพัฒนาโปรแกรมแบบส่วนติดต่อผู้ใช้ (GUI) โดยประกอบด้วย NumPy, SciPy, Matplotlib, Tkinter และ os

NumPy (import numpy as np) ใช้สำหรับการคำนวณเชิงตัวเลขและการจัดการข้อมูลแบบอาร์เรย์ รองรับการคำนวณฟังก์ชันทางคณิตศาสตร์ เช่น sine และ cosine รวมถึงช่วยเพิ่มประสิทธิภาพในการประมวลผลด้วยการคำนวณแบบเวกเตอร์

Matplotlib (import matplotlib.pyplot as plt) ใช้สำหรับการสร้างกราฟเพื่อแสดงผลเปรียบเทียบระหว่างฟังก์ชันต้นฉบับและอนุกรมฟูเรียร์ โดยสามารถกำหนดรูปแบบของกราฟได้อย่างหลากหลาย และทำงานร่วมกับ Tkinter ผ่าน FigureCanvasTkAgg เพื่อแสดงผลภายในหน้าต่างโปรแกรม

SciPy (from scipy.integrate import quad) ใช้สำหรับการหาค่าอินทิกรัลเชิงตัวเลข ซึ่งเป็นขั้นตอนสำคัญในการคำนวณสัมประสิทธิ์ของอนุกรมฟูเรียร์ โดยใช้วิธี adaptive quadrature ที่มีความแม่นยำสูง

Tkinter (import tkinter as tk และ from tkinter import ttk, messagebox) ใช้สำหรับพัฒนาโปรแกรมแบบ GUI โดย ttk ช่วยให้ส่วนติดต่อผู้ใช้มีความสวยงามและทันสมัยมากขึ้น และ messagebox ใช้สำหรับแสดงข้อความแจ้งเตือนหรือข้อผิดพลาด

os (import os) ใช้สำหรับจัดการคำสั่งในระดับระบบ โดยในโปรแกรมนี้ใช้สำหรับสั่งปิดโปรแกรมอย่างสมบูรณ์

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from scipy.integrate import quad
import tkinter as tk
from tkinter import ttk, messagebox
import os
```

2. เมธอด `__init__` และ `setup_ui` (การสร้างและเตรียมโครงสร้างพื้นฐานของโปรแกรม)

เมธอด `__init__` และ `setup_ui` เป็นส่วนสำคัญที่ทำหน้าที่กำหนดโครงสร้างหลักของโปรแกรม ทั้งในด้านการตั้งค่าเริ่มต้นและการออกแบบส่วนติดต่อผู้ใช้ (Graphical User Interface: GUI) เพื่อรองรับการทำงานของระบบคำนวณ Fourier Series

2.1 เมธอด `__init__` : การกำหนดสถานะเริ่มต้นของโปรแกรม

เมธอด `__init__` เป็นคอนสตรัคเตอร์ของคลาส ซึ่งจะถูกรับใช้ทันทีเมื่อมีการสร้างอ็อบเจกต์ของโปรแกรม โดยมีหน้าที่หลักดังนี้:

1. การอ้างอิงหน้าต่างหลัก

```
self.root = root
```

ใช้เก็บ reference ของหน้าต่างหลักจาก Tkinter เพื่อให้สามารถควบคุมองค์ประกอบต่าง ๆ ภายในโปรแกรมได้

2. การตั้งค่าคุณสมบัติของหน้าต่าง

```
self.root.title("Fourier series Calculator & Plotter")
```

```
self.root.geometry("1100x850")
```

- กำหนดชื่อโปรแกรมเพื่อสื่อถึงวัตถุประสงค์การใช้งาน
- กำหนดขนาดหน้าต่างให้เหมาะสมกับการแสดงทั้ง input และกราฟ

3. การจัดการเหตุการณ์ปิดโปรแกรม (Exit Handling)

```
self.root.protocol("WM_DELETE_WINDOW", self.on_closing)
```

กำหนดให้เมื่อผู้ใช้กดปิดหน้าต่าง ระบบจะเรียกเมธอด `on_closing` แทนการปิดทันที ซึ่งมีประโยชน์ในกรณีที่ต้อง:

- เคลียร์ทรัพยากร (resources)
- ป้องกันการปิดแบบไม่ตั้งใจ
- หรือบันทึกข้อมูลก่อนออกจากโปรแกรม

4. การเรียกสร้างส่วนติดต่อผู้ใช้

```
self.setup_ui()
```

เป็นการแยกหน้าที่ (separation of concerns) โดยให้ `__init__` ดูแลเฉพาะ “การตั้งค่าเริ่มต้น” และให้ `setup_ui` ดูแล “โครงสร้าง UI” โดยเฉพาะ

2.2 เมธอด `setup_ui` : การออกแบบและจัดวางส่วนติดต่อผู้ใช้

เมธอดนี้ทำหน้าที่สร้างองค์ประกอบทั้งหมดที่ผู้ใช้สามารถมองเห็นและโต้ตอบได้ โดยออกแบบในลักษณะ Modular เพื่อให้ใช้งานง่ายและพัฒนาได้สะดวก

1. การจัดกลุ่มด้วย `LabelFrame`

```
ctrl_frame = ttk.LabelFrame(...)
```

ใช้ `LabelFrame` เพื่อสร้างกรอบที่มีหัวข้อกำกับ (“Interval & Function Configuration”) ซึ่งช่วย:

- แยกส่วนการรับค่าพารามิเตอร์ออกจากส่วนอื่น
- เพิ่มความชัดเจนในการใช้งาน
- ทำให้ UI ดูเป็นระบบมากขึ้น

2. การรับค่าช่วงของฟังก์ชัน (Interval Input)

```
self.left_entry
```

```
self.right_entry
```

ใช้รับค่าขอบเขตซ้าย (a) และขอบเขตขวา (b) ของฟังก์ชัน โดย:

- กำหนดค่าเริ่มต้นเป็น $-\pi$ และ π
- ช่วยกำหนดช่วงสำหรับการคำนวณ Fourier Series

ค่าช่วงนี้มีผลโดยตรงต่อ:

- ความยาวคาบ (Period, T)
- และค่าคงที่ L ในสูตร Fourier

3. การเลือกโหมดฟังก์ชัน

```
self.mode_combo = ttk.Combobox(...)
```

รองรับ 2 รูปแบบ:

- Single Function $\rightarrow$ ฟังก์ชันเดียวตลอดช่วง
- Piecewise Function $\rightarrow$ ฟังก์ชันแบ่งช่วง

```
self.mode_combo.bind("<<ComboboxSelected>>", ...)
```

เมื่อผู้ใช้เปลี่ยนโหมด จะเรียก `toggle_mode()` เพื่อ:

- ปรับ UI แบบไดนามิก
- แสดงหรือซ่อน input ที่เกี่ยวข้องกับ Piecewise

4. การรับนิพจน์ของฟังก์ชัน

```
self.f1_entry
```

```
self.f2_entry
```

- $f_1(x) \rightarrow$ ฟังก์ชันหลัก
- $f_2(x) \rightarrow$ ใช้เฉพาะกรณี Piecewise

ผู้ใช้งานสามารถป้อนนิพจน์ทางคณิตศาสตร์ เช่น:

- $-x$
- x^{**2}
- $\sin(x)$

5. การกำหนดจำนวนพจน์ (Number of Terms)

`self.n_entry`

ใช้กำหนดจำนวนพจน์ (N) ใน Fourier Series:

- ค่ายิ่งมาก $\rightarrow$ ความแม่นยำสูงขึ้น
- แต่ใช้เวลาในการคำนวณมากขึ้น

6. การรับค่าจุดแบ่ง (Split Point)

`self.split_entry`

ใช้ในกรณี Piecewise เพื่อกำหนด:

- จุดที่ฟังก์ชันเปลี่ยนรูปแบบ

7. ปุ่มควบคุมการทำงาน

`ttk.Button(..., command=self.process)`

เมื่อผู้ใช้กดปุ่ม:

- จะเรียกเมธอด `process()`
- ซึ่งจะทำหน้าที่:
 - อ่านค่าจาก input

- คำนวณ Fourier Series
- แสดงผลลัพธ์ทั้งในรูปสมการและกราฟ

8. การแสดงผลสมการ

```
self.formula_box = tk.Text(...)
```

ใช้แสดงสมการ Fourier Series ที่คำนวณได้:

- ตั้งค่าเป็น state="disabled" เพื่อป้องกันการแก้ไข
- ใช้ font แบบ monospaced เพื่อให้อ่านสมการง่าย

9. การแสดงผลกราฟ (Matplotlib Integration)

```
self.fig, self.ax = plt.subplots(...)
self.canvas = FigureCanvasTkAgg(...)
```

เป็นการเชื่อมการทำงานระหว่าง:

- Matplotlib (สำหรับคำนวณและวาดกราฟ)
- Tkinter (สำหรับแสดงผลในหน้าต่างโปรแกรม)

คุณสมบัติสำคัญ:

- สามารถวาดกราฟภายใน GUI ได้โดยตรง
- รองรับการอัปเดตกราฟเมื่อข้อมูลเปลี่ยน
- ทำให้โปรแกรมมีลักษณะ interactive

10. การตั้งค่าโหมดเริ่มต้น

```
self.toggle_mode()
```

ใช้กำหนดสถานะเริ่มต้นของ UI ให้สอดคล้องกับโหมดที่เลือกไว้ (ค่าเริ่มต้นคือ Single)

เมธอด `__init__` และ `setup_ui` ทำงานร่วมกันเพื่อ:

- กำหนดค่าเริ่มต้นและพฤติกรรมของโปรแกรม
- ออกแบบและจัดวางอินเทอร์เฟซอย่างเป็นระบบ
- รองรับการรับข้อมูลจากผู้ใช้ในหลายรูปแบบ
- เตรียมพื้นที่สำหรับแสดงผลทั้งสมการและกราฟ
- และเชื่อมการทำงานระหว่าง GUI กับระบบคำนวณ

```
class FourierFinalApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Fourier series Calculator & Plotter")
        self.root.geometry("1100x850")
        self.root.protocol("WM_DELETE_WINDOW", self.on_closing)
        self.setup_ui()
```

```

def setup_ui(self):
    ctrl_frame = ttk.LabelFrame(self.root, text="1. Interval & Function Configuration ", padding=15)
    ctrl_frame.pack(fill="x", padx=20, pady=10)

    # ขอบเขต a, b (เพื่อหา T และ L)
    ttk.Label(ctrl_frame, text="ขอบเขตซ้าย (a):").grid(row=0, column=0)
    self.left_entry = ttk.Entry(ctrl_frame, width=12); self.left_entry.insert(0, "-pi")
    self.left_entry.grid(row=0, column=1, padx=5)

    ttk.Label(ctrl_frame, text="ขอบเขตขวา (b):").grid(row=0, column=2)
    self.right_entry = ttk.Entry(ctrl_frame, width=12); self.right_entry.insert(0, "pi")
    self.right_entry.grid(row=0, column=3, padx=5)

    self.mode_var = tk.StringVar(value="Single")
    self.mode_combo = ttk.Combobox(ctrl_frame, textvariable=self.mode_var, values=["Single", "Piecewise"], state="readonly", width=15)
    self.mode_combo.grid(row=0, column=5, padx=5)
    self.mode_combo.bind("<<ComboboxSelected>>", lambda e: self.toggle_mode())

    # ฟังก์ชัน
    self.lbl_f1 = ttk.Label(ctrl_frame, text="f1(x):")
    self.lbl_f1.grid(row=1, column=0, pady=10)
    self.f1_entry = ttk.Entry(ctrl_frame, width=30); self.f1_entry.insert(0, "-x")
    self.f1_entry.grid(row=1, column=1, columnspan=2, padx=5, sticky="w")

    ttk.Label(ctrl_frame, text="จำนวนพจน์ (N):").grid(row=1, column=3)
    self.n_entry = ttk.Entry(ctrl_frame, width=10); self.n_entry.insert(0, "4")
    self.n_entry.grid(row=1, column=4)

    # ส่วน Piecewise
    self.lbl_f2 = ttk.Label(ctrl_frame, text="f2(x):")
    self.f2_entry = ttk.Entry(ctrl_frame, width=30); self.f2_entry.insert(0, "x")
    self.lbl_split = ttk.Label(ctrl_frame, text="จุดแบ่ง (Split):")
    self.split_entry = ttk.Entry(ctrl_frame, width=10); self.split_entry.insert(0, "0")

    ttk.Button(ctrl_frame, text="Calculate & Plot (3 Periods)", command=self.process).grid(row=3, column=0, columnspan=6, pady=10)

    # กราฟ
    self.fig, self.ax = plt.subplots(figsize=(7, 4), dpi=100)
    self.canvas = FigureCanvasTkAgg(self.fig, master=self.root)
    self.canvas.get_tk_widget().pack(fill="both", expand=True, padx=20, pady=10)
    self.toggle_mode()

```

3. เมธอด toggle_mode (Dynamic UI Controller)

หน้าที่: ควบคุมสถานะการมองเห็นของ Widget ตามโหมดการใช้งาน Logic Analysis: เมธอดนี้ทำหน้าที่ตรวจสอบสถานะจากตัวแปร mode_var หากผู้ใช้เลือกโหมด Piecewise โปรแกรมจะเรียกใช้คำสั่ง .grid() เพื่อวางช่องใส่ฟังก์ชันที่ 2 และจุดแบ่ง (Split point) ออกมาทันที และจะใช้ .grid_remove() เพื่อซ่อนพารามิเตอร์เหล่านั้นเมื่อกลับสู่โหมดปกติ ช่วยลดความผิดพลาดในการป้อนข้อมูล (Input Validation)

```

def toggle_mode(self):
    if self.mode_var.get() == "Piecewise":
        self.lbl_f2.grid(row=2, column=0); self.f2_entry.grid(row=2, column=1, columnspan=2, padx=5, sticky="w")
        self.lbl_split.grid(row=2, column=3); self.split_entry.grid(row=2, column=4)
    else:
        self.lbl_f2.grid_remove(); self.f2_entry.grid_remove(); self.lbl_split.grid_remove(); self.split_entry.grid_remove()

```

4. เมธอด `f_eval` (Mathematical Interpreter)

หน้าที่: แปลง "ข้อความ" จากผู้ใช้ให้เป็น "ค่าคงที่คณิตศาสตร์" Sandboxing: โปรแกรมใช้ฟังก์ชัน `eval()` ในการตีความสมการ แต่มีการจำกัดขอบเขต (Namespace) อย่างเข้มงวด โดยอนุญาตให้เรียกใช้เฉพาะตัวแปร `x` และฟังก์ชันจาก NumPy เท่านั้น Error Prevention: เมธอดนี้ออกแบบมาเพื่อรองรับการรับค่าทั้งแบบตัวเลขเดี่ยว (Scalar) และแบบกลุ่มข้อมูล (Array) เพื่อความยืดหยุ่นในการนำไปคำนวณในอินทิเกรตและการวาดกราฟ

ฟังก์ชันนี้รับพารามิเตอร์ 2 ตัว คือ

- `x`: ค่าตัวแปรอิสระ (อาจเป็นตัวเลขหรือ array)
- `expr`: ข้อความนิพจน์ เช่น `"sin(x)"` หรือ `"x**2"`

ภายในฟังก์ชันใช้คำสั่ง `eval()` เพื่อแปลงข้อความให้กลายเป็นนิพจน์ทางคณิตศาสตร์และคำนวณผลลัพธ์ทันที เพื่อความปลอดภัย มีการจำกัดขอบเขตของ `eval` ดังนี้:

```
{"__builtins__": None}
```

เป็นการปิดการเข้าถึงคำสั่งพื้นฐานของ Python เพื่อลดความเสี่ยงจากการรันโค้ดที่ไม่ปลอดภัย

เป็นการกำหนดตัวแปรและฟังก์ชันที่ “อนุญาตให้ใช้ได้” เท่านั้น เช่น:

- `x` ใช้ในนิพจน์
- `sin(x)`, `cos(x)` สำหรับฟังก์ชันตรีโกณ
- `pi` สำหรับค่าคงที่
- `abs(x)` สำหรับค่าสัมบูรณ์
- `np` สำหรับการคำนวณเชิงตัวเลข

```
def f_eval(self, x, expr):
    return eval(expr, {"__builtins__": None}, {"x": x, "np": np, "pi": np.pi, "sin": np.sin, "cos": np.cos, "abs": np.abs})
```

5. เมธอด process คือ ส่วนประมวลผลกลางที่ทำหน้าที่เชื่อมโยงระหว่างการรับค่าจากผู้ใช้ (Input) การคำนวณทางคณิตศาสตร์ขั้นสูง (Processing) และการส่งผลลัพธ์ไปแสดงผล (Output) โดยสามารถแบ่งขั้นตอนการทำงานภายในอย่างละเอียดได้ดังนี้

5.1 การเตรียมพารามิเตอร์ทางคณิตศาสตร์ (Initialization & Constants)

ในขั้นแรก เมธอดจะดึงค่าจากช่องกรอกข้อมูล (Entry Widgets) มาทำการคำนวณค่าคงที่พื้นฐานที่จำเป็นต่อการหาอนุกรมฟูรีเยร์:

- **การดึงค่าขอบเขต:** ใช้ฟังก์ชัน `eval()` เพื่อแปลงข้อความจากช่อง `left_entry (a)` และ `right_entry (b)` ให้เป็นค่าตัวเลขจริง โดยรองรับการพิมพ์ค่าในรูปของ π หรือตัวเลขทศนิยม
- **การคำนวณคาบ (Period):** คำนวณ $T = b - a$ ซึ่งคือความยาวของหนึ่งรอบสัญญาณ
- **การคำนวณครึ่งคาบ (Half-period, L):** คำนวณ $L = \frac{T}{2}$ ซึ่งเป็นค่าวิกฤตที่ใช้ในสูตรของฟูรีเยร์เพื่อปรับขนาดของความถี่ในพจน์ Sine และ Cosine ให้เหมาะสมกับช่วงที่กำหนด

5.2 การคำนวณค่าสัมประสิทธิ์ a_0

ค่า a_0 คือค่าเฉลี่ยของฟังก์ชันบนหนึ่งคาบ (หรือพจน์คงที่) โดยโปรแกรมจะแยกการคำนวณตามโหมดที่เลือก

- สูตรทางคณิตศาสตร์

$$a_0 = \frac{1}{2L} \int_a^b f(x) dx$$

- **การทำงานของโปรแกรม:** เรียกใช้ฟังก์ชัน `quad` จาก Library SciPy เพื่อหาปริพันธ์เชิงตัวเลข (Numerical Integration) โดยส่งพารามิเตอร์เป็นฟังก์ชัน `f(x)` และขอบเขต `[a, b]` ผลลัพธ์ที่ได้จะถูกนำมาหารด้วย `2L` ตามสูตร

5.3 การคำนวณสัมประสิทธิ์ a_n และ b_n (Harmonic Iteration)

โปรแกรมจะทำการหาค่าความแรงของสัญญาณในแต่ละความถี่ (Harmonics) โดยใช้การวนลูป (Loop) ตั้งแต่พจน์ที่ $n = 1$ ไปจนถึง N ตามที่ผู้ใช้ระบุ:

- สูตรสัมประสิทธิ์ a_n :

$$a_n = \frac{1}{L} \int_a^b f(x) \cos\left(\frac{n\pi x}{L}\right) dx$$

- สูตรสัมประสิทธิ์ b_n :

$$b_n = \frac{1}{L} \int_a^b f(x) \sin\left(\frac{n\pi x}{L}\right) dx$$

การปรับใช้ใน Code

ภายในลูปจะมีการสร้าง **Lambda Function** (ฟังก์ชันชั่วคราว) เพื่อคูณฟังก์ชันต้นฉบับ $f(x)$ เข้ากับฟังก์ชันฐาน (Basis Function) คือ $\cos\left(\frac{n\pi x}{L}\right)$ หรือ $\sin\left(\frac{n\pi x}{L}\right)$ ก่อนจะส่งให้เมธอด `quad` ทำการอินทิเกรตเพื่อหาค่าสัมประสิทธิ์ในลำดับนั้น ๆ

5.4 ตรรกะการประมวลผลฟังก์ชันแบ่งช่วง (Piecewise Logic Handling)

ความโดดเด่นของเมธอดนี้คือการรองรับฟังก์ชันที่ไม่ต่อเนื่อง (Piecewise Functions) โดยใช้คุณสมบัติการรวมกันของการอินทิเกรต (Additivity Property):

- หากผู้ใช้เลือกโหมด **Piecewise** โปรแกรมจะรับค่าจุดแบ่ง (s)
- **กระบวนการคำนวณ:** แทนที่จะอินทิเกรตรวดเดียว โปรแกรมจะแยกการคำนวณออกเป็น 2 ช่วง คือ $[a, s]$ สำหรับฟังก์ชัน $f_1(x)$ และช่วง $[s, b]$ สำหรับฟังก์ชัน $f_2(x)$

- สูตรที่ประยุกต์ใช้:

$$\int_a^b \dots = \int_a^s f_1(x) \dots dx + \int_s^b f_2(x) \dots dx$$

- ผลรวมจากทั้งสองช่วงจะถูกนำมารวมกันและหารด้วย L เพื่อให้ได้ค่าสัมประสิทธิ์ที่ถูกต้องตามหลักการ

5.5 การส่งต่อข้อมูลเพื่อแสดงผล (Data Hand-off)

หลังจากที่วนลูปจนได้สัมประสิทธิ์ครบทุกพจน์และเก็บไว้ใน List (a_coeffs และ b_coeffs) แล้ว เมธอด process จะทำหน้าที่เป็นตัวส่งการให้ส่วนอื่นทำงานต่อ:

- เรียก display_formula: เพื่อนำสัมประสิทธิ์ไปจัดรูปแบบเป็นข้อความสมการที่มนุษย์อ่านเข้าใจ
- เรียก plot_result: เพื่อส่งข้อมูลพารามิเตอร์ทั้งหมดไปวาดกราฟเปรียบเทียบเชิงทัศนภาพ

5.6 ระบบป้องกันข้อผิดพลาด (Exception Handling)

กระบวนการทั้งหมดจะถูกห่อหุ้มด้วยบล็อก try...except เพื่อดักจับข้อผิดพลาดที่อาจเกิดขึ้น เช่น:

- ผู้ใช้ป้อนสูตรผิดไวยากรณ์ (Syntax Error)
- การหารด้วยศูนย์ (Division by Zero)
- ค่า N ไม่เป็นจำนวนเต็ม

หากเกิดข้อผิดพลาด โปรแกรมจะแสดง messagebox แจ้งเตือนผู้ใช้ทันทีแทนที่จะหยุดการทำงาน (Crash)

Algorithm (Pseudo Code) ในส่วนการ Process

INPUT a, b, function $f(x)$, N

COMPUTE $T = b - a$ COMPUTE $L = T / 2$

COMPUTE a_0 using integration

INITIALIZE arrays $a_n[]$, $b_n[]$

FOR $n = 1$ TO N:

 COMPUTE $a_n = \text{integral}(f(x) * \cos(...))$

 COMPUTE $b_n = \text{integral}(f(x) * \sin(...))$ STORE in arrays

END FOR

DEFINE Fourier function $F(x)$

FOR each x in range:

 CALCULATE original value

 CALCULATE Fourier value

END FOR

DISPLAY formula

PLOT both graphs

Code ในส่วน Process

```

def process(self):
    try:
        a = eval(self.left_entry.get(), {"np": np, "pi": np.pi})
        b = eval(self.right_entry.get(), {"np": np, "pi": np.pi})
        T = b - a
        L = T / 2 # ครึ่งคาบ L ตรงตามสูตรในรูป
        f1_str = self.f1_entry.get()
        N = int(self.n_entry.get())

        # 1. คำนวณสัมประสิทธิ์ตามสูตรในรูปภาพเป็นๆ
        if self.mode_var.get() == "Single":
            # a0 = 1/(2L) * integral
            a0, _ = quad(lambda x: self.f_eval(x, f1_str), a, b)
            a0 = a0 / (2 * L)

            a_coeffs, b_coeffs = [], []
            for n in range(1, N + 1):
                # an = 1/L * integral(f(x)*cos(n*pi*x/L))
                an, _ = quad(lambda x: self.f_eval(x, f1_str)*np.cos(n*np.pi*x/L), a, b)
                bn, _ = quad(lambda x: self.f_eval(x, f1_str)*np.sin(n*np.pi*x/L), a, b)
                a_coeffs.append(an/L); b_coeffs.append(bn/L)
        else:
            f2_str = self.f2_entry.get(); s = eval(self.split_entry.get(), {"np": np, "pi": np.pi})
            # a0 แยกสองช่วง
            a0_p1, _ = quad(lambda x: self.f_eval(x, f1_str), a, s)
            a0_p2, _ = quad(lambda x: self.f_eval(x, f2_str), s, b)
            a0 = (a0_p1 + a0_p2) / (2 * L)

            a_coeffs, b_coeffs = [], []
            for n in range(1, N + 1):
                an1, _ = quad(lambda x: self.f_eval(x, f1_str)*np.cos(n*np.pi*x/L), a, s)
                an2, _ = quad(lambda x: self.f_eval(x, f2_str)*np.cos(n*np.pi*x/L), s, b)
                bn1, _ = quad(lambda x: self.f_eval(x, f1_str)*np.sin(n*np.pi*x/L), a, s)
                bn2, _ = quad(lambda x: self.f_eval(x, f2_str)*np.sin(n*np.pi*x/L), s, b)
                a_coeffs.append((an1 + an2)/L); b_coeffs.append((bn1 + bn2)/L)

        # 2. แสดงสมการและวาดกราฟ
        self.display_formula(a0, a_coeffs, b_coeffs, L, N)
        self.plot_result(a, b, T, L, a0, a_coeffs, b_coeffs, f1_str, N)

    except Exception as e: messagebox.showerror("Error", str(e))

```

6. เมธอด display formula (Symbolic Formatter)

หน้าที่ เป็นส่วนการจัดการการแสดงผลจากผลลัพธ์การคำนวณ ให้เป็นในรูป "สมการอนุกรมฟูเรียร์"

- Precision Control: มีระบบการปัดเศษทศนิยม (Rounding) และการตรวจสอบค่าสัมประสิทธิ์ (Coefficient Checking) หากพจน์ใดมีค่าเข้าใกล้ศูนย์มากเกินไป โปรแกรมจะตัดพจน์นั้นทิ้งเพื่อความชัดเจนของสมการ
- Readable Output: มีการปรับแต่งสัญลักษณ์ เช่น เปลี่ยน pi เป็นสัญลักษณ์ที่อ่านง่าย และการจัดการเครื่องหมาย +/- อัตโนมัติ เพื่อให้ได้ผลลัพธ์ที่ใกล้เคียงกับในตำราวิชาการมากที่สุด

สมการอนุกรมฟูเรียร์ตรีโกณมิติ

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left[a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right) \right]$$

```
def display_formula(self, a0, a, b, L, N_limit):
    res = "f(x) = "
    count = 0
    if abs(a0) > 1e-4: res += f"{round(a0, 4)} "; count += 1

    for n in range(1, len(a) + 1):
        if count >= N_limit: break
        for coeff, func in [(a[n-1], "cos"), (b[n-1], "sin")]:
            if count < N_limit and abs(coeff) > 1e-4:
                sign = " + " if coeff > 0 else " - "
                if res == "f(x) = " and sign == " + ": sign = ""

                if np.isclose(L, np.pi):
                    inner = f"{n if n>1 else ''}x"
                else:
                    l_v = round(L, 2)
                    inner = f"{n}πx/{l_v}" if n > 1 else f"πx/{l_v}"

                res += f"{sign}{abs(round(coeff, 4))}{func}({inner}) "
                count += 1

    self.formula_box.config(state="normal"); self.formula_box.delete("1.0", tk.END)
    self.formula_box.insert("1.0", res + "+ ..."); self.formula_box.config(state="disabled")
```

7. เมธอด plot result (The Visualization Layer)

หน้าที่: สร้างกราฟการลู่เข้าของอนุกรมฟูเรียร์เทียบกับฟังก์ชันจริง

- Periodic Extension: ใช้สูตร $((x_i - a) \% T) + a$ เพื่อจำลองว่าสัญญาณนั้นเป็นสัญญาณคาบ (Periodic Signal) ที่เกิดขึ้นซ้ำๆ กันอย่างต่อเนื่อง
- Additive Synthesis: การสร้างเส้นกราฟสีแดงเกิดจากการรวมกันของคลื่นย่อยๆ (Summation) โดยเริ่มจาก a_0 แล้วบวกทับด้วย $a_n \cos\left(\frac{n\pi x}{L}\right)$ และ $b_n \sin\left(\frac{n\pi x}{L}\right)$ ทีละพจน์ ทำให้ผู้ใช้เห็นการเปลี่ยนแปลงจากคลื่นไซน์ธรรมดาๆ กลายเป็นรูปร่างของฟังก์ชันต้นฉบับเมื่อค่า N เพิ่มขึ้น

```
def plot_result(self, a, b, T, L, a0, a_coeffs, b_coeffs, f1_str, N):
    self.ax.clear()
    x_w = np.linspace(a - T, b + T, 1500)

    # ฟังก์ชันต้นฉบับ (เส้นประ)
    def periodic_f(xi):
        xw = ((xi - a) % T) + a
        if self.mode_var.get() == "Single": return self.f_eval(xw, f1_str)
        s = eval(self.split_entry.get(), {"np": np, "pi": np.pi})
        return self.f_eval(xw, f1_str if xw < s else self.f2_entry.get())

    y_o = [periodic_f(xi) for xi in x_w]

    # ฟังก์ชันฟูเรียร์ (เส้นแดง)
    y_f = np.full_like(x_w, a0)
    for n in range(1, N + 1):
        y_f += a_coeffs[n-1]*np.cos(n*np.pi*x_w/L) + b_coeffs[n-1]*np.sin(n*np.pi*x_w/L)

    self.ax.plot(x_w, y_o, 'k--', alpha=0.3, label="Original")
    self.ax.plot(x_w, y_f, 'r-', label=f"Fourier (N={N})")
    self.ax.axhline(0, color='black', lw=1); self.ax.grid(True, linestyle=':'); self.ax.legend(); self.canvas.draw()
```

8. เมธอด on_closing (System Resource Cleanup)

หน้าที่: จัดการหน่วยความจำและปิดโปรแกรมอย่างสมบูรณ์

System Insight: ในโปรแกรมที่มีการวาดกราฟและใช้ไลบรารีหลายตัว มักเกิดปัญหา "Zombie Process" หรือโปรแกรมที่ปิดไปแล้วแต่ยังค้างอยู่ในหน่วยความจำ เมธอดนี้จึงถูกเขียนมาเพื่อสั่ง `plt.close('all')` และ `os._exit(0)` เพื่อตัดการทำงานในระดับระบบปฏิบัติการทันทีเมื่อผู้ใช้กดปิดหน้าต่าง

```
def on_closing(self):
    plt.close('all'); self.root.quit(); self.root.destroy(); os._exit(0)
```

9. ส่วนรันโปรแกรม

ทำหน้าที่เป็น "จุดเริ่มต้นของการทำงาน" (Entry Point) เพื่อเปลี่ยนจากการนิยามคลาส (Definition) มาเป็นการทำงานจริงของแอปพลิเคชัน

- `if __name__ == "__main__":` เปรียบเสมือน "สวิตช์สตาร์ท" ที่บอกให้โปรแกรมเริ่มทำงานเฉพาะเมื่อเราเปิดไฟล์นี้โดยตรง ป้องกันไม่ให้โค้ดทำงานซ้ำซ้อนหากถูกนำไปเรียกใช้ในไฟล์อื่น
- `root = tk.Tk()` สร้างหน้าต่างหลัก (Main Window) ซึ่งเป็นพื้นฐานหรือ "ผืนผ้าใบ" สำหรับจัดวางองค์ประกอบทั้งหมดของโปรแกรม
- `app = FourierFinalApp(root)` เป็นการนำหน้าต่างหลักที่สร้างขึ้น ไปประกอบเข้ากับชุดคำสั่งและตรรกะที่เราเขียนไว้ในคลาส เพื่อเริ่มต้นการสร้างปุ่มและระบบคำนวณ
- `root.mainloop()` เริ่มต้นวงจรการตอบสนอง (Event Loop) สั่งให้หน้าต่างโปรแกรมแสดงผลค้างไว้ และคอยเฝ้าดูการกระทำของผู้ใช้ (เช่น การกดปุ่มหรือพิมพ์ข้อมูล) เพื่อให้โปรแกรมโต้ตอบได้ตลอดเวลา

```
if __name__ == "__main__":
    root = tk.Tk(); app = FourierFinalApp(root); root.mainloop()
```

ขั้นตอนการใช้งานโปรแกรม (User Manual)

เพื่อให้ผู้ใช้งานสามารถวิเคราะห์หอนุกรมฟูเรียร์ได้อย่างแม่นยำ โปรแกรมถูกออกแบบให้มีลำดับการใช้งานที่เรียบง่ายตามขั้นตอนดังนี้:

1. การเริ่มต้นโปรแกรม (Startup)

- ทำการรันไฟล์ Python (.py) ผ่าน Terminal หรือ IDE (เช่น VS Code, PyCharm)
- หน้าต่างโปรแกรม **Fourier series Calculator & Plotter**
- จะปรากฏขึ้น พร้อมค่าเริ่มต้น (Default) เป็นฟังก์ชัน $f(x) = -x$ บนช่วง $[-\pi, \pi]$

2. การกำหนดขอบเขตและคาบ (Setting Boundaries)

- **ขอบเขตซ้าย (a):** ป้อนค่าจุดเริ่มต้นของช่วง เช่น $-\pi$ หรือ -2
- **ขอบเขตขวา (b):** ป้อนค่าจุดสิ้นสุดของช่วง เช่น π หรือ 2
- **หมายเหตุ:** โปรแกรมจะนำค่า $b - a$ มาคำนวณหาคาบ (T) และครึ่งคาบ (L) โดยอัตโนมัติเพื่อให้สอดคล้องกับทฤษฎี

3. การเลือกโหมดการทำงาน (Mode Selection)

- เลือกจากช่อง Dropdown Menu:
 - **Single:** สำหรับฟังก์ชันต่อเนื่องพจน์เดียวตลอดช่วง $[a, b]$
 - **Piecewise:** สำหรับฟังก์ชันแบ่งช่วง (เช่น ฟังก์ชันขั้นบันได) โดยโปรแกรมจะเปิดช่องสำหรับใส่ฟังก์ชันที่ 2 ($f_2(x)$) และจุดแบ่ง (Split Point) เพิ่มเติม

4. การป้อนสมการคณิตศาสตร์ (Function Input)

- ป้อนฟังก์ชันในรูปแบบไวยากรณ์ Python เช่น:
 - ใช้ x^{**2} แทน x^2
 - ใช้ $\text{np.sin}(x)$, $\text{np.cos}(x)$, $\text{np.exp}(x)$ สำหรับฟังก์ชันตรีโกณมิติและเอ็กซ์โพเนนเชียล
 - ใช้ np.pi หรือ π สำหรับค่า π

5. การกำหนดความละเอียด (Setting N)

- ป้อนจำนวนพจน์ (N) ที่ต้องการให้โปรแกรมคำนวณ
- **ข้อแนะนำ:** ค่า N ที่มากขึ้นจะช่วยให้กราฟอนุกรมฟูเรียร์เข้าใกล้รูปร่างของฟังก์ชันต้นฉบับมากขึ้น (แต่จะใช้เวลาประมวลผลนานขึ้นเล็กน้อย)

6. การประมวลผลและวาดกราฟ (Execution & Visualization)

- คลิกปุ่ม "Calculate & Plot (3 Periods)"
- โปรแกรมจะทำการอินทิเกรตหาค่าสัมประสิทธิ์ a_0, a_n, b_n และแสดงผลลัพธ์ 2 ส่วนพร้อมกัน:
 1. **Formula Box:** แสดงสมการอนุกรมฟูเรียร์ที่คำนวณได้ในรูปแบบสัญลักษณ์
 2. **Graph Area:** แสดงกราฟเปรียบเทียบระหว่างฟังก์ชันต้นฉบับ (เส้นประสีดำ) และอนุกรมฟูเรียร์ (เส้นแดง) โดยวาดต่อเนื่องกัน 3 คาบเพื่อให้เห็นความเป็นคาบ (Periodicity)

7. การอ่านและวิเคราะห์ผลลัพธ์ (Result Analysis)

- **สังเกตสมการ:** ตรวจสอบค่าสัมประสิทธิ์ในช่องข้อความเพื่อนำไปใช้ในการคำนวณเชิงทฤษฎี
- **สังเกตกราฟ:** ดูการลู่เข้า (Convergence) ของสัญญาณ

ข้อควรระวัง: หากป้อนรูปแบบสมการผิด โปรแกรมจะแสดงหน้าต่างแจ้งเตือน (Error Message) ให้ตรวจสอบไวยากรณ์อีกครั้ง เพื่อป้องกันโปรแกรมหยุดการทำงาน

การจำลองทดสอบโจทย์ตัวอย่าง

1.

$$f(x) = \begin{cases} -1; -\pi \leq x < 0 \\ 1; 0 \leq x < \pi \end{cases}$$

โดยที่คาบมีค่าเท่ากับ $2L = 2\pi$

เนื่องจาก $f(x)$ เป็นฟังก์ชันคี่ ดังนั้น $a_0 = a_n = 0$

$$\begin{aligned} b_n &= \frac{1}{\pi} \int_{-\pi}^0 (-1) \sin(nx) dx + \frac{1}{\pi} \int_0^{\pi} (1) \sin(nx) dx \\ &= \frac{1}{n\pi} - \frac{\cos(n\pi)}{n\pi} - \frac{\cos(n\pi)}{n\pi} - \frac{1}{n\pi} \\ &= \frac{2}{n\pi} - \frac{2\cos(n\pi)}{n\pi} \\ b_n &= \frac{2}{n\pi} (1 - \cos(n\pi)) \end{aligned}$$

ดังนั้นอนุกรมฟูเรียร์จะได้

$$f(x) = a_0 + \sum_{n=1}^{\infty} [a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right)]$$

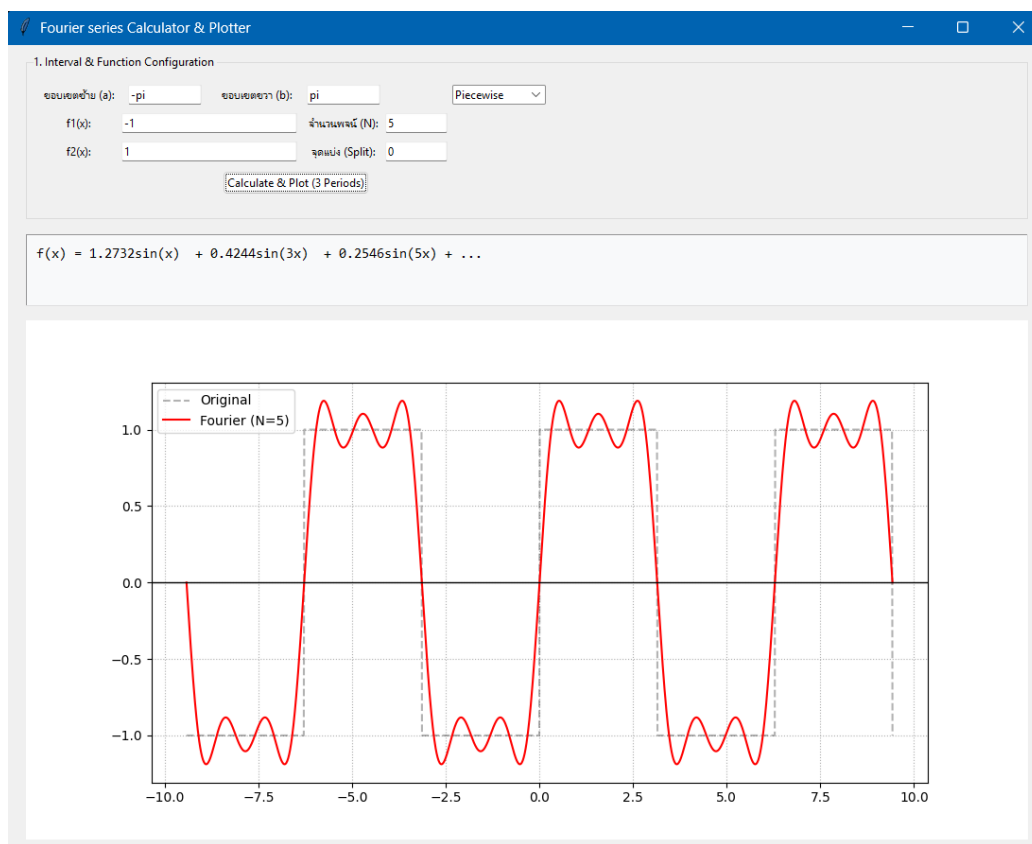
$$a_0 = a_n = 0$$

$$f(x) = \sum_{n=1}^{\infty} [b_n \sin\left(\frac{n\pi x}{L}\right)]$$

$$f(x) = b_1 \sin(x) + b_2 \sin(2x) + b_3 \sin(3x) + \dots$$

$$f(x) = \frac{4}{\pi} \sin(x) + \frac{4}{3\pi} \sin(3x) + \frac{4}{5\pi} \sin(5x) + \dots$$

โปรแกรมจำลอง



ดังนั้นคำตอบที่โปรแกรมจำลองหาได้นั้นเท่ากับการคำนวณ

2.

$$f(x) = \begin{cases} -x; -1 \leq x < 0 \\ x; 0 \leq x < 1 \end{cases}$$

$$f(x) = f(x+2)$$

โดยที่คาบมีค่าเท่ากับ $2L = 2$

เนื่องจาก $f(x)$ เป็นฟังก์ชันคี่ ดังนั้น $b_n = 0$

$$\begin{aligned} a_0 &= \frac{1}{2L} \int_{-L}^L f(x) dx = \frac{1}{2} \int_{-1}^0 (-x) dx + \frac{1}{2} \int_0^1 x dx \\ &= \left[0 - \left(-\frac{1}{4} \right) \right] + \left[\frac{1}{4} + 0 \right] \\ a_0 &= \frac{1}{2} \end{aligned}$$

$$\begin{aligned} a_n &= \frac{1}{L} \int_{-L}^L f(x) \cos\left(\frac{n\pi x}{L}\right) dx \\ &= \left(\frac{1}{1} \int_{-1}^0 (-x) \cos\left(\frac{n\pi x}{1}\right) dx \right) + \left(\frac{1}{1} \int_0^1 x \cos\left(\frac{n\pi x}{1}\right) dx \right) \\ &= - \left[\left(-\sin \frac{n\pi}{n\pi} \right) + \left(\frac{1}{n^2 \pi^2} - \frac{\cos(n\pi)}{n^2 \pi^2} \right) \right] + \left[\left(\sin \frac{n\pi}{n\pi} \right) + \left(\frac{\cos(n\pi)}{n^2 \pi^2} - \frac{1}{n^2 \pi^2} \right) \right] \\ a_n &= \frac{2 \sin(n\pi)}{n\pi} + \frac{2 \cos(n\pi)}{n^2 \pi^2} - \frac{2}{n^2 \pi^2} = \frac{2}{n^2 \pi^2} (\cos(n\pi) - 1) \end{aligned}$$

ดังนั้นอนุกรมฟูเรียร์จะได้

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left[a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right) \right]$$

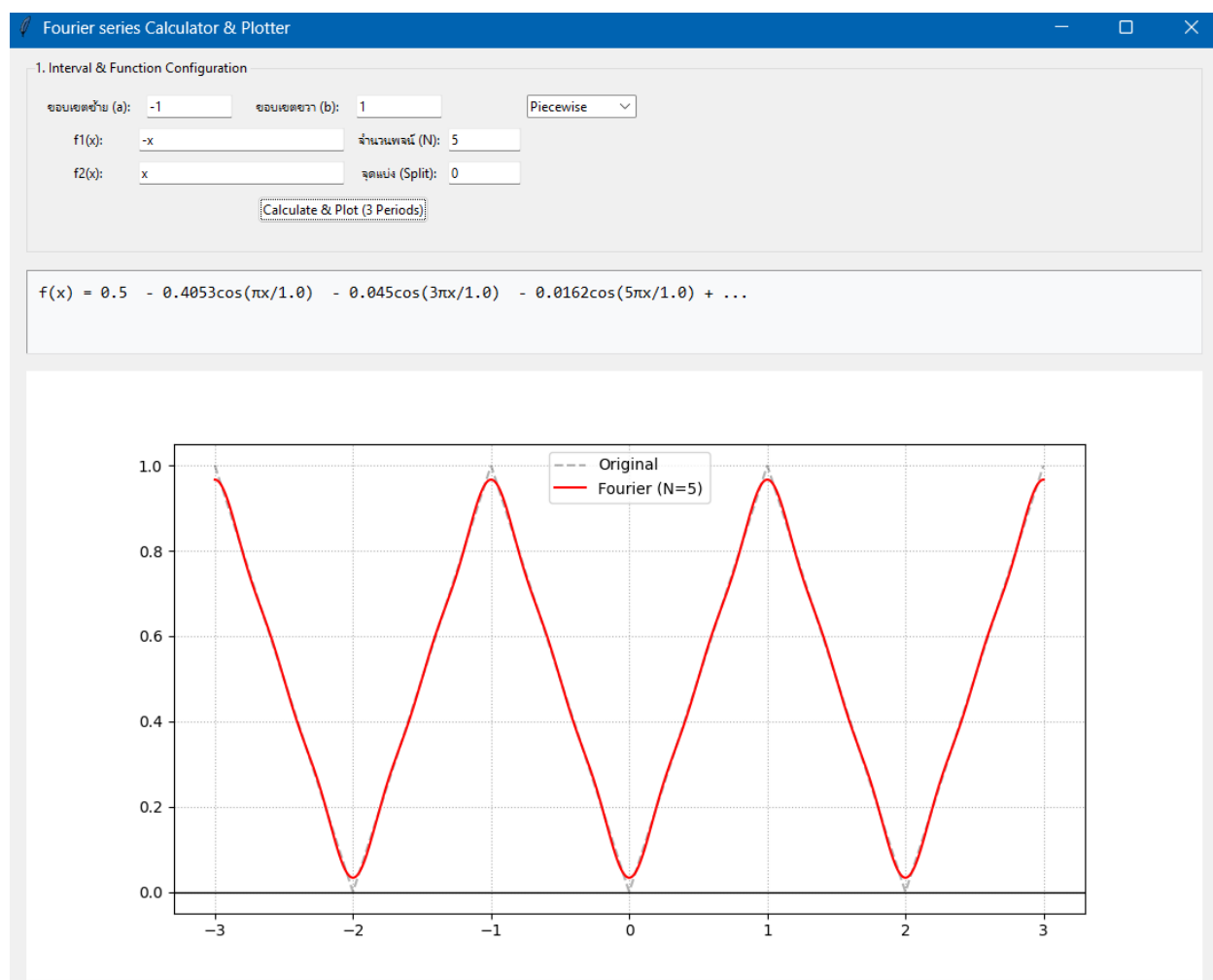
$$b_n = 0$$

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left[a_n \cos \left(\frac{n\pi x}{L} \right) \right]$$

$$f(x) = a_0 + a_1 \cos(x\pi) + a_2 \cos(2x\pi) + a_3 \cos(3x\pi)$$

$$f(x) = \frac{1}{2} - \frac{4}{\pi^2} \cos(x\pi) - \frac{4}{9\pi^2} \cos(3x\pi) - \frac{4}{25\pi^2} \cos(5x\pi)$$

โปรแกรมจำลอง



ดังนั้นคำตอบที่โปรแกรมจำลองหาได้นั้นเท่ากับการคำนวณ

3.

$$f(x) = x, -\pi \leq x \leq \pi$$

$$f(x) = f(x + 2\pi)$$

โดยที่คาบมีค่าเท่ากับ $2L = 2\pi$

เนื่องจาก $f(x)$ เป็นฟังก์ชันคี่ ดังนั้น $a_0 = a_n = 0$

$$\begin{aligned} b_n &= \frac{1}{L} \int_{-L}^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx \\ &= \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(nx) dx \end{aligned}$$

$$b_n = \frac{1}{\pi} \int_{-\pi}^{\pi} x \sin(nx) dx$$

พิจารณา $\int_{-\pi}^{\pi} x \sin(nx) dx$ จากการอินทิเกรตทีละส่วน

จะได้
$$\int_{-\pi}^{\pi} x \sin(nx) dx = \frac{x \cos(nx)}{n} + \frac{\sin(nx)}{n^2}$$

$$\begin{aligned} b_n &= \int_{-\pi}^{\pi} x \sin(nx) dx \\ &= \frac{1}{\pi} \left[\left(-\left(\frac{\pi \cos(n\pi)}{n} - \frac{(-\pi) \cos(n\pi)}{n} \right) \right) + \left(\frac{\sin(n\pi)}{n^2} - \frac{\sin(-n\pi)}{n^2} \right) \right] \\ &= -\frac{1}{\pi} \left(\frac{\pi \cos(n\pi)}{n} + \frac{\pi \cos(n\pi)}{n} \right) = -\frac{1}{\pi} \left(\frac{2\pi \cos(n\pi)}{n} \right) \\ b_n &= -\frac{2 \cos(n\pi)}{n} \end{aligned}$$

ดังนั้นอนุกรมฟูเรียร์จะได้

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left[a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right) \right]$$

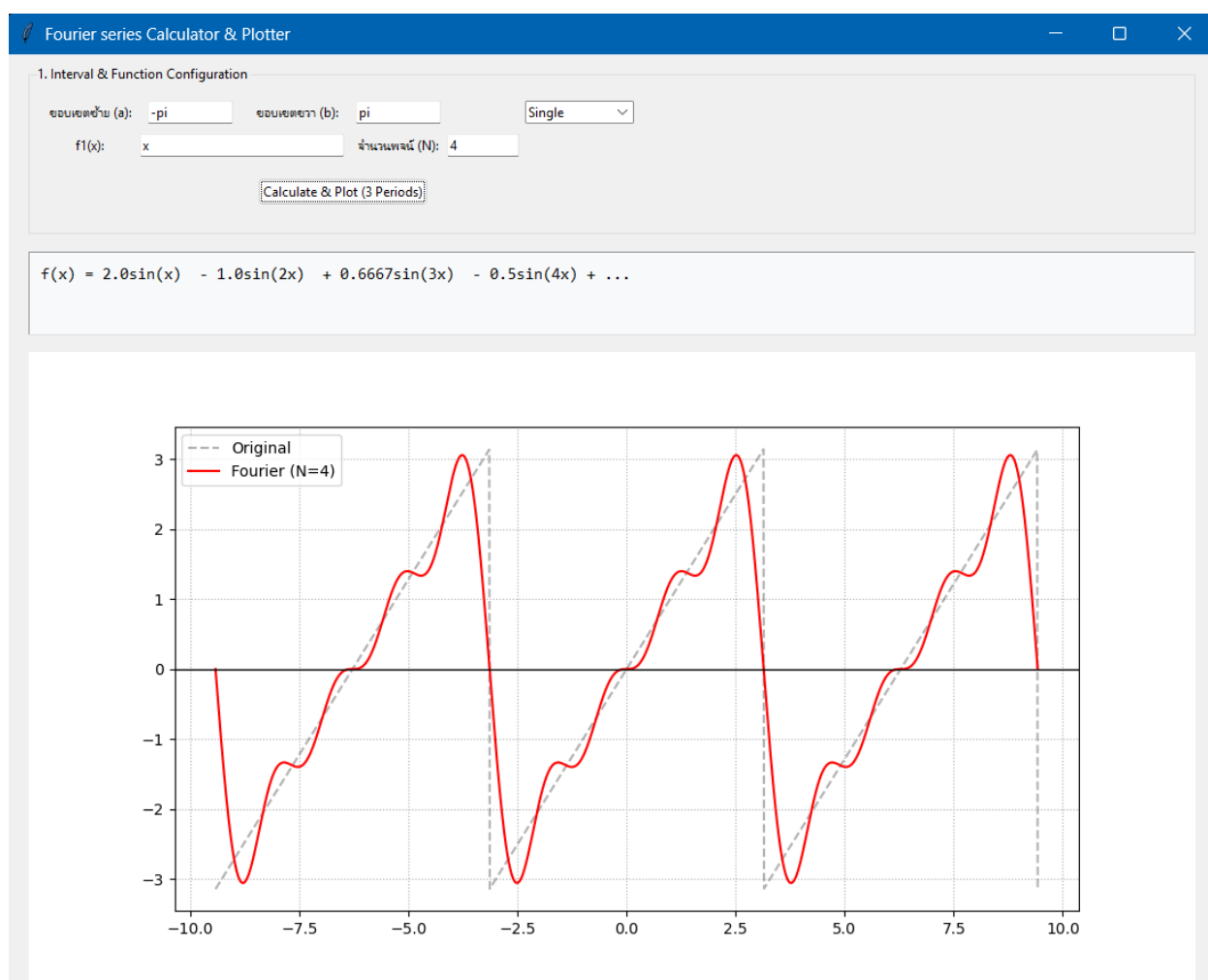
$$a_0 = a_n = 0$$

$$f(x) = \sum_{n=1}^{\infty} [b_n \sin(\frac{n\pi x}{L})]$$

$$f(x) = b_1 \sin(x) + b_2 \sin(2x) + b_3 \sin(3x) + \dots$$

$$f(x) = 2 \sin(x) - \sin(2x) + \frac{2}{3} \sin(3x) - \frac{\sin(4x)}{2} + \dots$$

โปรแกรมจำลอง



ดังนั้นคำตอบที่โปรแกรมจำลองหาได้นั้นเท่ากับการคำนวณ

4. $f(x) = 4 - x^2, -2 \leq x \leq 2$

$$f(x) = f(x+4)$$

- คาบ $T = 4 \Rightarrow 2L = 4 \Rightarrow L = 2$
- ฟังก์ชัน $4 - x^2$ เป็น ฟังก์ชันคู่ (even)

ดังนั้น:

- ไม่มี sine $\rightarrow b_n = 0$ ใช้เฉพาะ cosine

รูปแบบอนุกรมฟูรีเยร์

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left[a_n \cos \left(\frac{n\pi x}{L} \right) + b_n \sin \left(\frac{n\pi x}{L} \right) \right]$$

พิจารณาสมมาตรฟังก์ชัน $f(x) = 4 - x^2$ เป็น ฟังก์ชันคู่

ดังนั้น

$$b_n = 0$$

คำนวณสัมประสิทธิ์

1. หา a_0

$$a_0 = \frac{1}{2L} \int_{-L}^L f(x) dx = \frac{1}{4} \int_{-2}^2 (4 - x^2) dx$$

$$= \frac{1}{4} \left[4x - \frac{x^3}{3} \right]_{-2}^2$$

$$\begin{aligned}
 &= \frac{1}{4} \left[\left(8 - \frac{8}{3}\right) - \left(-8 + \frac{8}{3}\right) \right] \\
 &= \frac{1}{4} \left(16 - \frac{16}{3}\right) = \frac{1}{4} \cdot \frac{32}{3} = \frac{8}{3}
 \end{aligned}$$

2. หา a_n

$$a_n = \frac{1}{L} \int_{-L}^L (4 - x^2) \cos \left(\frac{n\pi x}{L} \right) dx$$

แทน $L = 2$

$$a_n = \frac{1}{2} \int_{-2}^2 (4 - x^2) \cos \left(\frac{n\pi x}{2} \right) dx$$

ใช้สมบัติฟังก์ชันคู่ อินทิเกรตเฉพาะ 0 ถึง 2 แล้วคูณ 2:

$$a_n = \int_0^2 (4 - x^2) \cos \left(\frac{n\pi x}{2} \right) dx$$

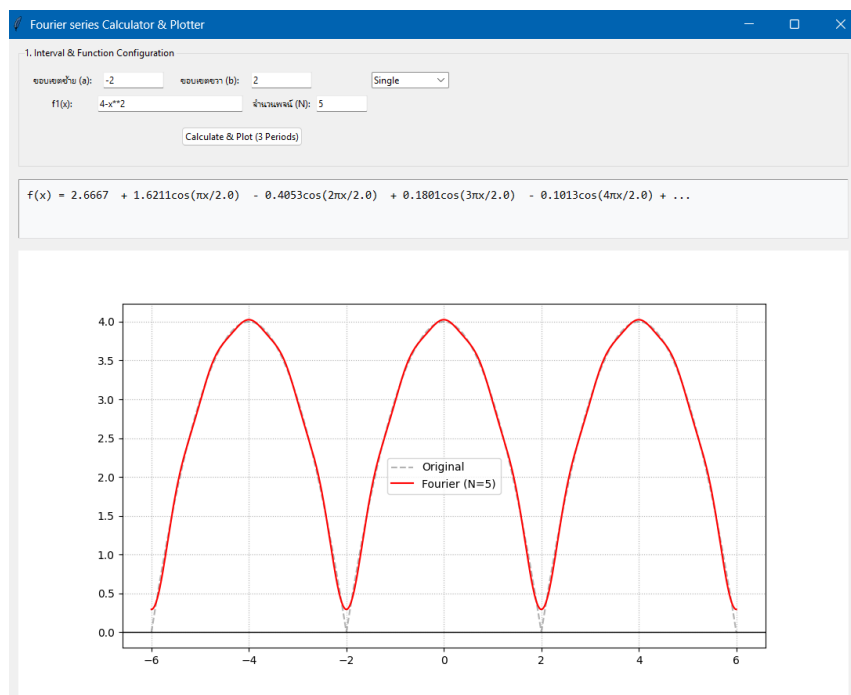
จะได้

$$a_n = \frac{16}{n^2 \pi^2} (-1)^n$$

ดังนั้นอนุกรมฟูรีเยร์จะได้

$$f(x) = \frac{8}{3} + \sum_{n=1}^{\infty} \frac{16}{n^2 \pi^2} (-1)^n \cos \left(\frac{n\pi x}{2} \right)$$

$$f(x) = \frac{8}{3} + \frac{16}{\pi^2} \cos\left(\frac{\pi x}{2}\right) - \frac{4}{\pi^2} \cos\left(\frac{2\pi x}{2}\right) + \frac{2}{\pi^2} \cos\left(\frac{3\pi x}{2}\right) + \dots$$



ดังนั้นคำตอบที่โปรแกรมจำลองหาได้นั้นเท่ากับการคำนวณ

5. $f(x) = 2 - x, -1 \leq x \leq 1 \rightarrow f(x) = f(x+2)$

$$f(x) = 2 - x, -1 \leq x \leq 1, \text{และคาบ } T = 2$$

ดังนั้น

$$2L = 2 \Rightarrow L = 1$$

$$f(x) = a_0 + \sum_{n=1}^{\infty} [a_n \cos(n\pi x) + b_n \sin(n\pi x)]$$

หา a_0

$$\begin{aligned} a_0 &= \frac{1}{2L} \int_{-L}^L f(x) dx \\ &= \frac{1}{2} \int_{-1}^1 (2-x) dx \\ &= \frac{1}{2} \left[2x - \frac{x^2}{2} \right]_{-1}^1 \\ &= \frac{1}{2} (4) \\ &= 2 \end{aligned}$$

หา a_n

$$a_n = \int_{-1}^1 (2-x) \cos(n\pi x) dx$$

แยกเป็น:

$$2\cos(n\pi x) > \text{even}$$

$$x\cos(n\pi x) \text{ odd} > \text{อินทิเกรต} = 0$$

ดังนั้น:

$$a_n = 2 \int_{-1}^1 \cos(n\pi x) dx$$

$$= 2 \left[\frac{\sin(n\pi x)}{n\pi} \right]_{-1}^1 = 0$$

หา b_n

$$b_n = \int_{-1}^1 (2-x) \sin(n\pi x) dx$$

แยก:

$$2\sin > \text{odd} = 0$$

$$x\sin > \text{even} = \text{ใช้ได้}$$

$$b_n = - \int_{-1}^1 x \sin(n\pi x) dx = -2 \int_0^1 x \sin(n\pi x) dx$$

อินทิเกรตโดย parts:

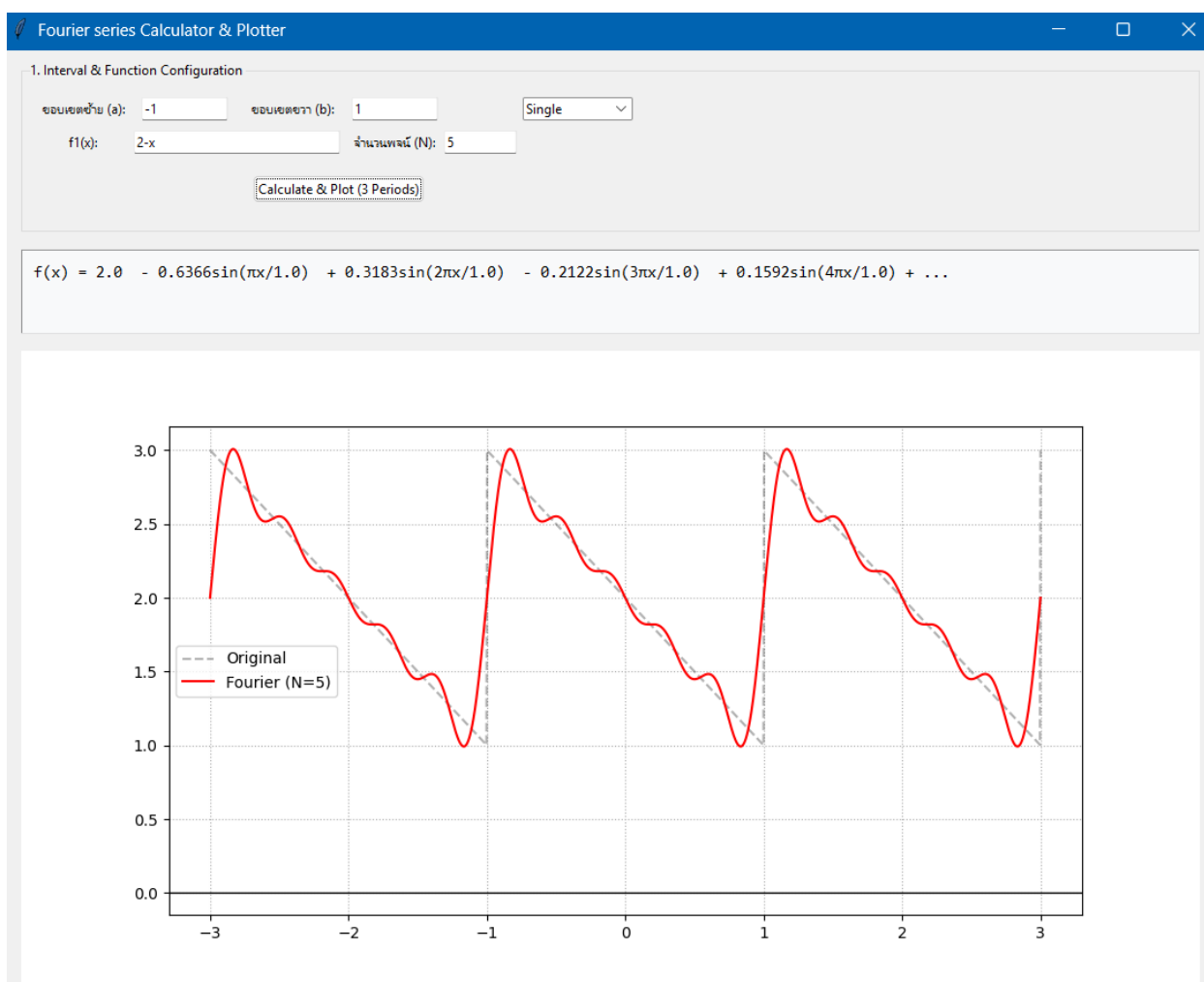
ได้ผลลัพธ์

$$b_n = \frac{2(-1)^n}{n\pi}$$

ดังนั้นอนุกรมฟูเรียร์จะได้

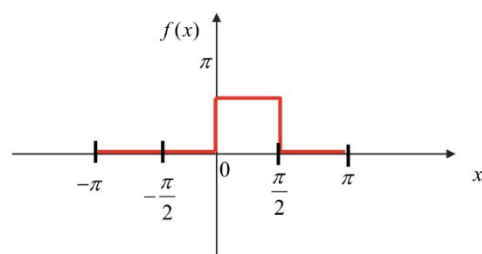
$$f(x) = 2 + \sum_{n=1}^{\infty} \frac{2(-1)^n}{n\pi} \sin(n\pi x)$$

$$f(x) = 2 + \frac{2}{\pi} \sin(\pi x) - \frac{2}{2\pi} \sin(2\pi x) + \frac{2}{3\pi} \sin(3\pi x) + \dots$$



ดังนั้นคำตอบที่โปรแกรมจำลองหาได้นั้นเท่ากับการคำนวณ

6.



จากรูปสามารถเขียนได้ว่า

$$f(x) = \begin{cases} \pi, & 0 < x < \frac{\pi}{2} \\ 0, & \text{ที่เหลือใน } (-\pi, \pi) \end{cases}$$

และเป็นฟังก์ชันคาบ 2π

$$f(x) = a_0 + \sum_{n=1}^{\infty} [a_n \cos(nx) + b_n \sin(nx)]$$

หา a_0

$$a_0 = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) dx$$

มีค่าเฉพาะช่วง $0 > \frac{\pi}{2}$:

$$\begin{aligned} a_0 &= \frac{1}{2\pi} \int_0^{\pi/2} \pi dx \\ &= \frac{1}{2\pi} \cdot \pi \cdot \frac{\pi}{2} \\ &= \frac{\pi}{4} \end{aligned}$$

หา a_n

$$\begin{aligned}
 a_n &= \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos(nx) \, dx \\
 &= \frac{1}{\pi} \int_0^{\pi/2} \pi \cos(nx) \, dx \\
 &= \int_0^{\pi/2} \cos(nx) \, dx \\
 &= \left[\frac{\sin(nx)}{n} \right]_0^{\pi/2} \\
 &= \frac{\sin\left(\frac{n\pi}{2}\right)}{n}
 \end{aligned}$$

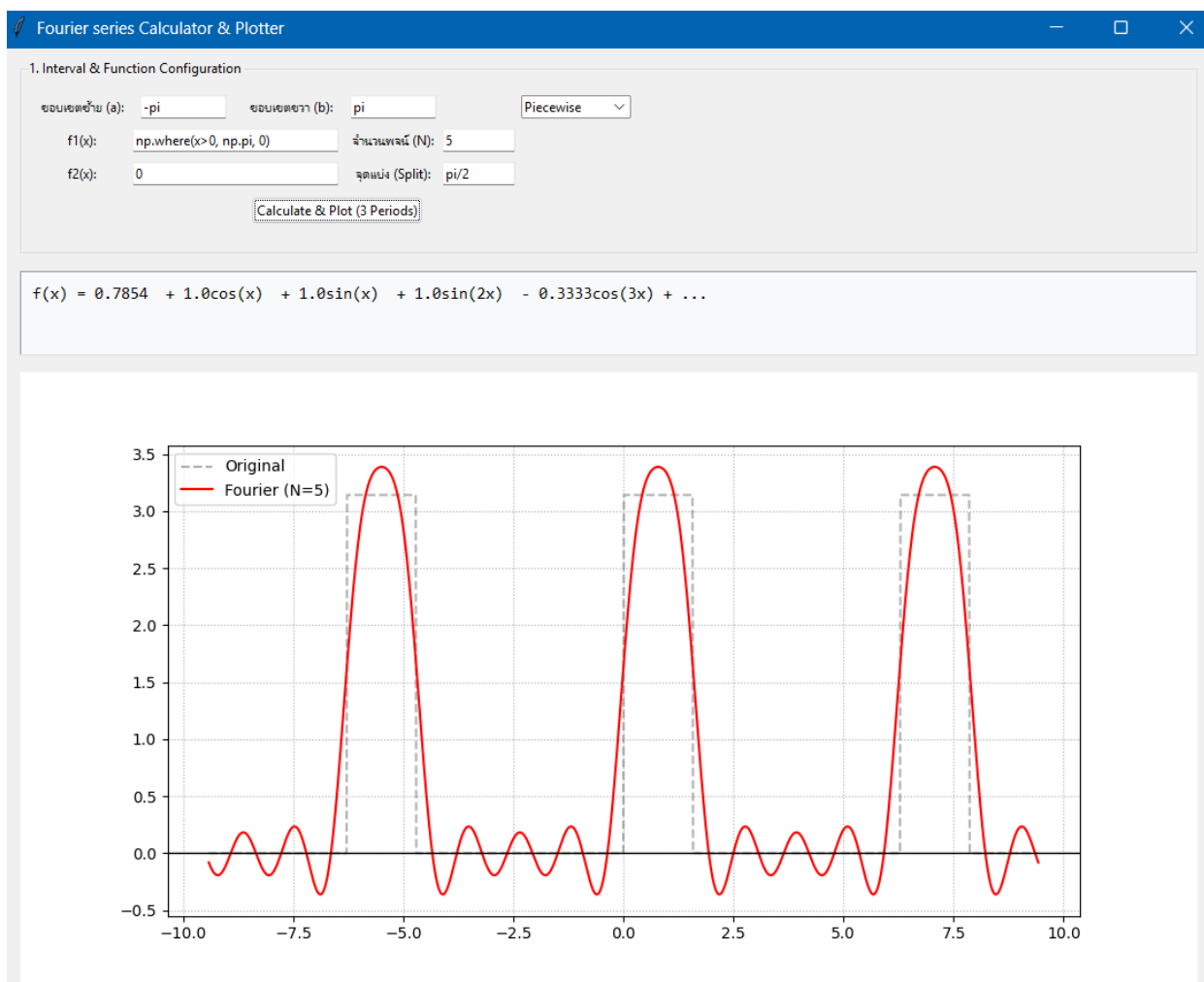
หา b_n

$$\begin{aligned}
 b_n &= \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(nx) \, dx \\
 &= \int_0^{\pi/2} \sin(nx) \, dx \\
 &= \left[\frac{-\cos(nx)}{n} \right]_0^{\pi/2} \\
 &= \frac{1 - \cos\left(\frac{n\pi}{2}\right)}{n}
 \end{aligned}$$

ดังนั้นอนุกรมฟูเรียร์จะได้

$$f(x) = \frac{\pi}{4} + \sum_{n=1}^{\infty} \left[\frac{\sin\left(\frac{n\pi}{2}\right)}{n} \cos(nx) + \frac{1 - \cos\left(\frac{n\pi}{2}\right)}{n} \sin(nx) \right]$$

$$f(x) = \frac{\pi}{4} + \cos(x) + \sin(x) + \sin(2x) - \frac{1}{3}\cos(3x) + \frac{1}{3}\sin(3x) + \dots$$



ดังนั้นคำตอบที่โปรแกรมจำลองหาได้นั้นเท่ากับการคำนวณ

ในข้อที่ 6 แตกต่างจากข้ออื่น เนื่องจากเป็นฟังก์ชันที่มี 3 ช่วง จึงจำเป็นต้องใช้คำสั่งจาก NumPy เข้ามาช่วยกำหนดเงื่อนไขของฟังก์ชัน

ในกรณีที่ฟังก์ชันมีมากกว่า 2 ช่วง การใช้เงื่อนไขแบบ if-else หลายชั้นอาจทำให้โค้ดมีความซับซ้อนมากขึ้น ดังนั้นจึงสามารถใช้ np.where() เพื่อช่วยเลือกสมการของฟังก์ชันในแต่ละช่วงได้อย่างเป็นระบบ โดยช่วงของตัวแปร x ถูกกำหนดจากขอบเขต a และ b ที่ผู้ใช้กำหนดไว้แล้ว จากนั้น np.where() จะตรวจสอบว่าค่า x อยู่ในช่วงใด และเลือกสมการที่สอดคล้องกับช่วงนั้นโดยอัตโนมัติ จากตัวอย่างข้อที่ 6 ใช้คำสั่ง np.where($x > 0$, np.pi, 0) ในฟังก์ชัน $f_1(x)$ เพื่อกำหนดให้ช่วงที่ $x < 0$ มีค่าเท่ากับ 0 และช่วงที่ $x > 0$ มีค่าเท่ากับ π กล่าวคือหากเงื่อนไข $x > 0$ เป็นจริง โปรแกรมจะเลือกค่า π แต่ถ้าเงื่อนไขเป็นเท็จ โปรแกรมจะเลือกค่า 0 แทน

ดังนั้น การใช้ np.where() ในข้อนี้จึงช่วยให้สามารถกำหนดฟังก์ชันแบบหลายช่วงได้อย่างชัดเจน ลดความซับซ้อนของโค้ด และทำให้การคำนวณรวมถึงการพล็อตกราฟเป็นไปอย่างถูกต้องมากขึ้น

สรุปการดำเนินงาน

1. การพัฒนาโปรแกรมคำนวณ Fourier Series ด้วยภาษา Python ได้มีการพัฒนาโปรแกรมด้วยภาษา Python โดยใช้ไลบรารี Tkinter สำหรับสร้างส่วนติดต่อผู้ใช้ และใช้ SciPy ในการคำนวณอินทิกรัลเพื่อหาค่าสัมประสิทธิ์ของอนุกรมฟูเรียร์ (a_0 , a_n , b_n) พร้อมทั้งใช้ Matplotlib ในการแสดงผลกราฟ
2. การทำความเข้าใจแนวคิดการแปลงฟังก์ชันเป็น sine และ cosine และการลู่เข้า (convergence) โปรแกรมสามารถแปลงฟังก์ชันที่ผู้ใช้ป้อนให้อยู่ในรูปของอนุกรม sine และ cosine ได้อย่างถูกต้อง และสามารถปรับจำนวนพจน์ (N) เพื่อสังเกตพฤติกรรมการลู่เข้าของอนุกรมฟูเรียร์เมื่อจำนวนพจน์เพิ่มขึ้น
3. การแสดงผลเชิงภาพเพื่อเปรียบเทียบฟังก์ชันจริงกับค่าประมาณ
โปรแกรมสามารถแสดงกราฟเปรียบเทียบระหว่างฟังก์ชันต้นฉบับและผลลัพธ์จาก Fourier Series ได้อย่างชัดเจน ทำให้ผู้ใช้สามารถมองเห็นความแตกต่างและความแม่นยำของการประมาณค่าได้ในเชิงภาพ

ข้อจำกัดของโปรแกรม

1. โปรแกรมรองรับเฉพาะฟังก์ชันที่อยู่ในรูปนิพจน์ทางคณิตศาสตร์พื้นฐาน เช่น $\sin$, $\cos$, abs และ numpy functions เท่านั้น
2. ความแม่นยำของผลลัพธ์ขึ้นอยู่กับจำนวนพจน์ (N) หากเลือกค่าน้อยเกินไปจะทำให้ผลลัพธ์คลาดเคลื่อน
3. การคำนวณด้วย numerical integration อาจใช้เวลาเพิ่มขึ้นเมื่อจำนวนพจน์มากหรือฟังก์ชันซับซ้อน
4. รองรับการใช้งานในรูปแบบ GUI เท่านั้น ไม่สามารถใช้งานผ่าน command line หรือ API ได้โดยตรง

ข้อเสนอแนะในการพัฒนาต่อ

1. เพิ่มฟังก์ชันสำเร็จรูป (preset functions) เพื่อให้ผู้ใช้เลือกใช้งานได้ง่ายขึ้น
2. เพิ่มระบบปรับความละเอียดของกราฟ (resolution control) เพื่อเพิ่มความแม่นยำในการแสดงผล
3. เพิ่มการ export ผลลัพธ์เป็นไฟล์ เช่น PDF หรือรูปภาพ เพื่อใช้ในรายงานหรือการเรียนการสอน
4. ปรับปรุง UI ให้มีความทันสมัยและใช้งานง่ายขึ้น เช่น ใช้ custom tkinter หรือ PyQt

บรรณานุกรม

BYJU'S. (n.d.). *Fourier series*. Retrieved from

<https://byjus.com/maths/fourier-series/>

Chulalongkorn University. (n.d.). *Fourier series (Chapter 10)*. Retrieved from

http://pioneer.netserve.chula.ac.th/~tdumrong/2301312/file_sheet_2555_2nd/312ch10_2555_2nd.pdf

JetBrains. (2024). *PyCharm Documentation*. Retrieved from

<https://www.jetbrains.com/pycharm/documentation/>

Microsoft. (2024). *Visual Studio Code Documentation*. Retrieved from

<https://code.visualstudio.com/docs>

Matplotlib Developers. (2024). *Matplotlib Documentation*. Retrieved from

<https://matplotlib.org/stable/contents.html>

Python Software Foundation. (2024). *Python Documentation*. Retrieved from

<https://docs.python.org/3/>

Scikit-learn Developers. (2024). *Scikit-learn documentation*. Retrieved from [https://scikit-](https://scikit-learn.org/stable/documentation.html)

[learn.org/stable/documentation.html](https://scikit-learn.org/stable/documentation.html)